

浙江大学



控制与决策系统仿真

课程大作业报告

多无人机协同搜索决策方法对比 与避障系统仿真研究

姓名：

学号：

课程： 控制与决策系统仿真

日期： 2026 年 6 月

浙江大学

目录

1 研究目的	1
2 研究内容	1
3 研究过程	2
3.1 系统总体方案	2
3.2 硬件平台与电路方案	2
3.3 软件程序框图	4
3.4 三种决策方法与安全约束	6
3.5 仿真设计与结果	7
3.6 从仿真到实机的实施路线	15
3.7 可行性与风险分析	16
4 研究体会	17
5 研究收获	17
6 建议	18
参考文献	18
附录 1：源代码	20
A.1 主入口与配置	20
A.2 地图与空间辅助函数	22
A.3 Q-learning 训练与评价	25
A.4 动作选择与环境更新	31
A.5 扩展实验与结果输出	36
附录 2：PaperPass 查重报告	46

1 研究目的

本报告研究多无人机在障碍环境中的协同搜索与避障决策问题。巡检、灾后搜救和区域侦察都希望在有限时间内扩大覆盖范围，单架无人机却受续航和感知距离限制。增加机数以后，重复搜索与航迹冲突随之出现，通信延迟还可能使原有分工失效。本文先比较任务层的决策方法，再讨论决策结果如何经过局部规划和飞控转化为实际飞行。

本次工作的直接目的是比较不同任务决策方法在多机搜索中的表现。在同一栅格环境中，三架无人机需要避开固定障碍物和其他机体，尽量访问更多未搜索区域。全局贪心、分区贪心和多智能体 Q-learning 是三种主要研究对象，随机策略只作为下限参照。实验从覆盖率、完成率、路径长度、冲突尝试、安全干预率、重复搜索率和机间最小距离等指标考察三种方法的差别。仿真并非课题终点，而是在接入真实感知、规划和飞控模块之前，对决策规则进行低成本排错的第一步。

仿真结果还需要落实到实机接口。三种方法只在任务层输出局部目标点或搜索子区域，不直接给出电机转速。离散目标先由局部规划器转换为满足速度、加速度和碰撞约束的连续轨迹，再交给 PX4 完成位置和姿态控制。统一目标接口后，上层方法可以直接切换，下游仍使用同一套飞控和 EGO-Planner[1] 等局部规划模块。

2 研究内容

MATLAB 二维栅格仿真是本报告的主体，其中比较全局贪心、分区贪心和 Q-learning，随机策略提供下限参照。程序完成地图生成、可达区域计算、多机同步运动和安全检查。障碍密度与机群规模实验用于检验三种方法在条件变化后的表现；Q-learning 另做奖励消融和局部观测扰动，以查找其性能受限的原因。实机部分尚未开展，报告给出的是二维决策器接入真实系统时所需的接口和测试路线。

仿真建立 20×20 栅格地图，使用固定障碍块与随机障碍点混合构造绕行、窄通道和局部遮挡。固定障碍包含五段横墙：(6–8 行, 5–14 列) 占据地图中部主要通道，(13–15 行, 7–17 列) 在下半区形成长条形隔断，(10–11 行, 2–8 列) 与 (4–5 行, 16–18 列) 分别在左侧和右上角制造局部遮挡，(17–18 行, 4–6 列) 在下边界附近组成小障碍群；此外按 0.05 的附加密度随机生成障碍格。三架无人机从不同位置出发，每步从上、下、左、右和悬停中选择一个动作。当候选位置越界、处于障碍格或与其他机体重合时，动作被拒绝并计入惩罚。访问新格子获得正奖励，重复访问、碰撞风险和机间距离过小则引入负奖励。

实机系统中，离散栅格动作不能直接作为飞行指令。本文将其重新解释为任务层的“下一局部搜索目标”，由局部规划器计算无碰的三维轨迹。感知与状态估计拟采

用 LiDAR、IMU 与激光惯性里程计，FAST-LIO2 的直接点云配准方法可作为这一环节的参考实现 [2]；环境以局部三维占据栅格表示。每架无人机保留本地规划和紧急停车能力，只广播必要的位姿、局部轨迹和已分配任务，以降低对持续通信的依赖。

三种方法使用的信息和分工方式不同。全局贪心读取整张覆盖地图，分区贪心先固定各机负责区域，Q-learning 则根据局部状态选择动作。本实验比较它们的覆盖效率，也记录路径代价、重复访问和安全干预。得到这些结果后，才能确定实机初期采用哪一种上层策略，以及是否需要保留切换接口。

3 研究过程

3.1 系统总体方案

系统采用“感知与估计-建图-任务决策-局部规划-飞行控制”的分层架构。EGO-Swarm 的去中心化系统为多机独立规划与局部轨迹交换提供了可参考的实现思路 [3]。本方案中，每架无人机独立运行状态估计、建图和局部规划，机间只交换任务区域与短时预测轨迹。这样即使短时丢包，单机仍能依靠本地地图停车或绕障，不会因中心节点中断而完全失去控制。

在这个架构中，全局贪心、分区贪心和 Q-learning 都位于慢时标的任务决策层，只输出搜索方向、目标点或任务区域。轨迹规划器根据实时占据地图计算平滑、无碰且满足动力学约束的轨迹；PX4 控制回路以更高频率跟踪位置、速度和姿态指令，其嵌入式软件框架与模块划分可参考 Meier 等人的设计 [4]。这种分层保留了统一的规划和控制链路，三种上层方法可以在相同下游条件下比较。

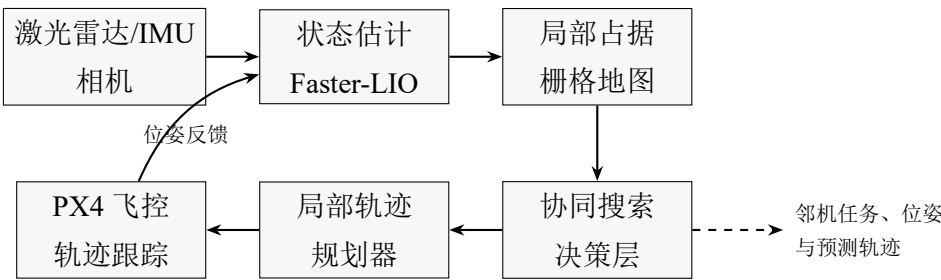


图 1 实机自主飞行系统架构

3.2 硬件平台与电路方案

实机方案拟采用四旋翼飞行平台作为载体，搭载 Jetson Orin Nano 作为机载计算单元，运行 Ubuntu、ROS、激光惯性里程计、建图、任务决策和轨迹规划；采用兼容 PX4 的飞控板完成姿态估计、位置控制和电机混控；三维感知以 Livox MID-360 类激光雷达与 IMU 为主，相机用于之后的目标识别或语义搜索。

表 1 实机系统的主要硬件与作用

模块	参考选型	主要作用及设计注意事项
飞行平台	四旋翼平台	提供机架、动力系统与防护结构；实际起飞前需重新测量总质量、重心与推重比
飞行控制	NxtPX4v2 或同类 PX4 飞控	读取 IMU、气压计等传感器，完成姿态、位置和电机回路，提供失联与低电压保护
机载计算	Jetson Orin Nano	运行定位、建图、规划和多机决策；需独立稳压、散热和上电时序管理
环境感知	MID-360 类 LiDAR	输出点云与图像，安装后需标定雷达-IMU
通信	5 GHz Wi-Fi 与 2.4 GHz 遥控	Wi-Fi 承载 ROS 状态和邻机轨迹；独立遥控链路用于人工接管和紧急停机
供电	6S 锂电池、电源模块与 DC-DC	动力与计算负载分支供电，共地并加入过流、欠压、反接和瞬态抑制

供电电路的核心原则是动力回路与弱电计算回路分支，避免电机快速变流导致机载计算机复位或传感器异常。6S 电池通过电源分配板向四路 ESC 供电，另外经隔离或低纹波 DC-DC 生成飞控、机载计算机与传感器各自需要的稳压电源。稳压值和连续电流应依照最终器件手册确定，不能仅根据标称功率选型；应为雷达上电浪涌、Jetson 计算峰值与散热风扇保留余量。

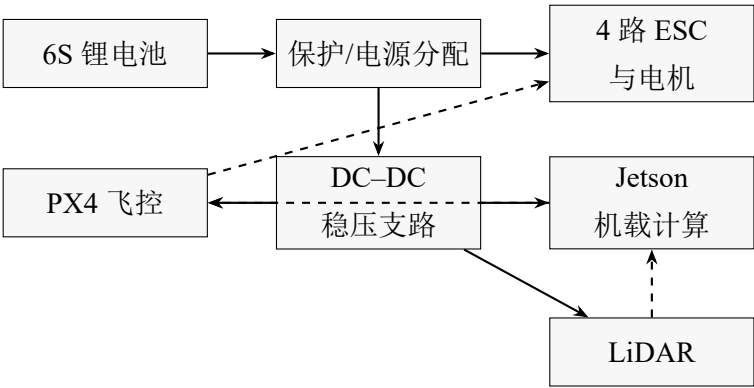


图 2 实机电源与主要信号连接框图

信号连接方面，机载计算机与飞控通过专用串口或 USB 传输 MAVLink 消息，雷达通过以太网或厂商规定接口输出点云，相机经 USB 输出图像。高电流电机线与传感器线束应分开布线，雷达、IMU 安装位置需兼顾视野、振动与重心。飞行前必须核对共地、接口电平和极性，并在不安装螺旋桨的条件下先完成上电和通信测试。

电源选型不能只根据电池容量估算续航。应先使用功率分析仪分别测量飞控、机

载计算机、雷达在启动、空闲与满载时的电流，再以各支路峰值之和乘以安全系数选择 DC-DC 模块。若计算机与雷达共用一路稳压，雷达上电瞬间可能使输出电压跌落，因此更稳妥的做法是独立分支并在负载端就近去耦。电源模块还应向飞控提供经标定的电压与电流采样，使低电压返航或降落逻辑依据的是实际电量，而不是机载计算机的软件估计。

对机载电路还需考虑电磁兼容与机械可靠性。ESC 和电机相线中的高频开关电流可能通过共阻抗、辐射或磁场耦合干扰 IMU、磁力计和通信链路。布局时应缩小大电流回路面积，让电源线与回流线紧邻，传感器线束避开 ESC 和大电流线。插头应有防反接和机械锁定，线束在进入机载计算机前需要应力释放，否则飞行振动会将插头松动变成间歇性故障。这些问题在纯软件仿真中不会出现，却直接决定实机系统能否长时间稳定运行。

3.3 软件程序框图

MATLAB 程序采用“主入口加功能包”的结构。`run_multi_uav_sim.m` 只负责读取运行选项、组织训练与评价流程；`+multi_uav` 目录中的文件分别处理参数配置、地图生成、三种决策方法、同步冲突裁决、扩展实验、数据导出和绘图。主入口共 65 行，按执行顺序组织，便于按需定位各模块而不必在千行脚本中查找。

程序首先生成地图和起点。全局贪心与分区贪心直接根据当前覆盖状态计算动作，Q-learning 则先以 ϵ -greedy 规则训练 Q 表。每个时间步中，所有无人机根据同一旧状态选择动作，再集中处理同目标格、对向交换和占用格冲突，最后同时更新位置与覆盖状态；训练阶段另外更新 Q 值。三种方法和随机参照均在同一组评估地图上运行，程序统一生成 CSV 指标、PNG 图像与可选 GIF 动画。主要文件之间的职责见表2。

表 2 MATLAB 程序模块及职责

文件或模块	职责
run_multi_uav_sim.m	程序入口，组织基准实验、扩展实验、输出与动画
defaultConfig.m、 buildMap.m	集中管理参数、起点、固定障碍和可达区域
trainQ*.m	单种子训练、并行多种子训练和参数实验中的顺序训练
evaluate*.m	在固定评估地图上运行策略并汇总统计量
chooseAction.m、 stepAgents.m	策略动作选择、同步冲突裁决、奖励与覆盖状态更新
runExperiments.m	障碍密度、机群规模、奖励消融和通信异常实验
writeOutputs.m、 plotBaseline.m、 animateSearch.m	CSV、静态图像和动态过程输出

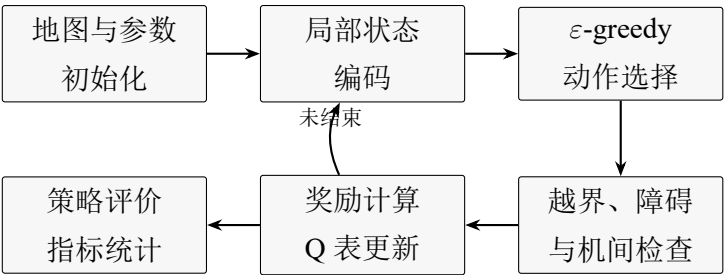


图 3 当前 MATLAB 仿真程序框图

向实机迁移时，上述 MATLAB 模块仍需改写为 ROS 节点。地图节点接收点云与位姿并维护占据栅格；任务决策节点以较低频率选择前沿区域或局部目标；轨迹规划节点在地图更新、轨迹冲突或目标改变时重规划；MAVROS 或对应桥接节点将轨迹指令发送给 PX4。日志节点应同步记录位姿、地图、规划时延、通信丢包和安全事件，以便事后分析。

任务决策节点不能在每次地图刷新后立即更换目标，否则连续跳变的目标会迫使规划器频繁重规划。接口中应设置到达容差和最短目标保持时间；无人机进入目标邻域后，任务层才选择下一目标。如果目标被新障碍占用、等待超时、局部规划连续失败或安全状态机触发，也要允许提前撤销目标。规划器尚未完成本次重规划时，任务层不再重复发送同一轮更新。

软件接口必须统一坐标系、时间戳和数据有效期。建议将世界系、里程计系、机体系和传感器系通过 TF 树显式表达，任务决策只读取带时间戳的位姿和地图版本。

如果邻机轨迹超过规定时间没有更新，不应继续把它当作准确预测，而应扩大安全区域或降低速度。同样，当定位协方差过大、点云时间戳滞后或局部地图长时间未更新时，规划器应拒绝生成高速轨迹。通过这些数据新鲜度检查，可以把“消息仍然存在”与“消息仍然可信”区分开。

多机信息交换也应控制数据量。不必在无线网中持续传输每架无人机的完整高分辨率点云，可优先广播自身位姿、有限时域的多项式轨迹、当前分区标识和少量前沿信息。前沿结构与分层探索规划可参考 FUEL 的处理方式 [5]，但多机任务分配仍需在本课题中单独设计。对于发送频率高但可容忍少量丢失的状态消息，应避免因反复重传堵塞网络；对于任务分配和模式切换等关键事件，则需要序号、确认和超时重发。

3.4 三种决策方法与安全约束

第 k 步时，第 i 架无人机的离散位置记为 $p_i(k) = [x_i(k), y_i(k)]$ ，三种方法共用动作集 $\mathcal{A} = \{\text{上, 下, 左, 右, 悬停}\}$ 。越界或进入障碍物的动作在决策前被排除，多个无人机电选动作后，再由同步冲突裁决处理同目标格、对向交换和占用格冲突。因此，三种方法的差别只在上层动作选择，不来自不同的运动或安全规则。

全局贪心在全部可达未访问格中寻找距离候选位置最近的目标。对动作 a 对应的候选位置 p_i^a ，评分写为

$$J_i(a) = 3I_{\text{new}} - 0.08\|g_i(a) - p_i^a\|_1 - 2I_{\text{near}}, \quad (1)$$

其中 $g_i(a)$ 是全局最近未访问格， I_{new} 表示候选格尚未访问， I_{near} 表示候选位置与邻机的曼哈顿距离不超过 1。分区贪心沿用同一评分，但目标只从本机负责的区域中选择。区域归属由栅格到三架无人机初始位置的距离确定；本机分区已经搜索完时，再退回全局最近未访问格。前者直接追求全局覆盖效率，后者用固定分工减少多机在同一区域内竞争。

Q-learning 把四邻域障碍物、四邻域未访问标记、邻机接近风险以及最近未访问格的方向写入状态，并采用经典更新规则 [6]：

$$Q(s, a) \leftarrow Q(s, a) + \alpha \left[r + \gamma \max_{a'} Q(s', a') - Q(s, a) \right], \quad (2)$$

其中 α 为学习率， γ 为折扣因子。奖励函数将新覆盖奖励、碰撞惩罚、重复访问惩罚、步长代价和距离引导项组合，使得“快速扩大覆盖”和“保持安全间隔”在同一决策函数中被考虑。

本实验采用共享参数的独立 Q-learning。三架无人机根据各自局部状态查询同一张 Q 表，先同时选取动作，再由集中安全层裁决冲突，最后分别用自身奖励更新共享参数。程序没有构造联合动作价值，也没有集中式 critic。独立学习者在多智能体环境中的协作与非平稳问题已有较早讨论 [7]；共享参数虽然减少了样本需求，其他

无人机不断变化的行为仍会改变单机所见的转移过程，因此不能直接套用单智能体 Q-learning 的收敛结论。

单步奖励写为

$$r_i = 2.5I_{\text{new}} - 8I_{\text{obs}} - 5I_{\text{conflict}} - 0.45I_{\text{repeat}} - 0.12I_{\text{hover}} - 1.2I_{\text{near}} - 0.03 + 0.12(d_{\text{before}} - d_{\text{after}}) - 0.003d_{\text{after}}, \quad (3)$$

其中 I 为相应事件的示性变量， d 为到本机分区内最近未访问格的曼哈顿距离。学习率 $\alpha = 0.18$ ，折扣因子 $\gamma = 0.92$ ，探索率由 0.55 线性降低至 0.05；每次训练 1200 回合，每回合最多 260 步。状态共 13 位，包括四邻域障碍、四邻域未访问标记、邻机接近标记和四个目标方向标记，因此 Q 表包含 $2^{13} \times 5$ 个状态-动作对。

这里需要区分地图大小和状态编码。只扩大地图而保持这 13 位局部状态不变，Q 表规模仍为 $2^{13} \times 5$ ；问题在于单机更难从局部信息判断大范围搜索进度。若继续加入动态障碍、更多邻机信息或更大的观测邻域，编码位数才会增加，表格规模也会随之快速增长。后续可分别考察以 DQN 近似离散动作价值，或采用 MAPPO 一类集中训练、分散执行的方法，但这些方法需要重新设计观测与训练流程，当前实验并未验证其效果。

实机安全不能仅依靠奖励函数。本方案将安全分为三层：任务层通过区域分工和轨迹交换减少冲突；规划层对障碍物和邻机轨迹设置安全裕量，并在冲突时重规划；飞控层实施地理围栏、最大速度、最大倾角、低电压保护、失联悬停或降落。另外保留独立遥控器与硬件停机权限，确保上层程序异常时仍可人工接管。

3.5 仿真设计与结果

三种方法的基准比较

基准实验以全局贪心、分区贪心和 Q-learning 为并列研究对象，随机策略只用于给出无目标搜索的下限。三种方法使用相同地图、起点、动作约束和同步冲突裁决。评价时记录覆盖率、完成率、完成步数、总路径、冲突尝试、安全干预率、重复搜索率和机间最小距离，不以单一指标决定优劣。

表 3 基准仿真指标（均值 $\pm$ 标准差）

策略	覆盖率/%	完成率/%	完成步数	路径长度	重复率/%	最小机间距离/栅格
随机	46.82 $\pm$ 5.99	0	260.0 $\pm$ 0.0	592.4 $\pm$ 11.9	82.71 $\pm$ 2.18	2.828 $\pm$ 2.398
全局贪心	88.96 $\pm$ 12.21	44	209.4 $\pm$ 59.4	384.2 $\pm$ 106.7	54.40 $\pm$ 18.37	1.000 $\pm$ 0.000
分区贪心	82.52 $\pm$ 16.26	24	234.7 $\pm$ 48.0	396.4 $\pm$ 154.5	63.37 $\pm$ 16.08	1.721 $\pm$ 1.020
Q-learning	56.25 $\pm$ 7.49	0	260.0 $\pm$ 0.0	581.8 $\pm$ 95.6	73.18 $\pm$ 8.26	1.561 $\pm$ 0.619

为降低单一地图带来的偶然性，正式试验统一使用 25 张评价地图。全局贪心和分区贪心在给定地图上的决策是确定的；Q-learning 包含训练随机性，因此用 3 个随机种子分别训练 1200 回合，再把得到的 3 张 Q 表放到同一组地图上评价。同一地图上的 3 次 Q-learning 结果先取平均，统计时仍以 25 张地图为独立评价单位。三种方法共用地图、起点和评价口径。

地图生成后，程序从全部无人机起点进行四邻域洪水搜索，只把能够从至少一个起点到达的自由格计入覆盖率分母。基准 25 张地图中，可达自由格占全部自由格的 $99.945\% \pm 0.162\%$ 。若直接以全部自由格为分母，随机障碍隔离出的少量栅格也会进入 96% 完成阈值，造成无法完成的目标。因此，本实验以可达自由格定义覆盖率和完成率。

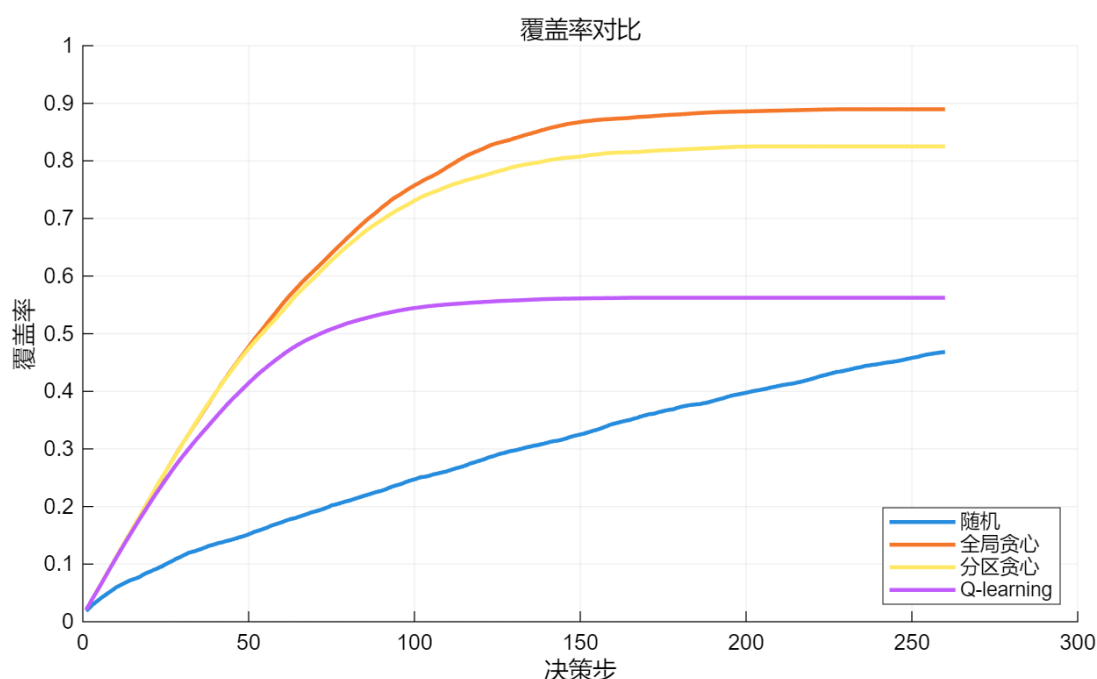


图 4 基准条件下四种策略的平均覆盖过程

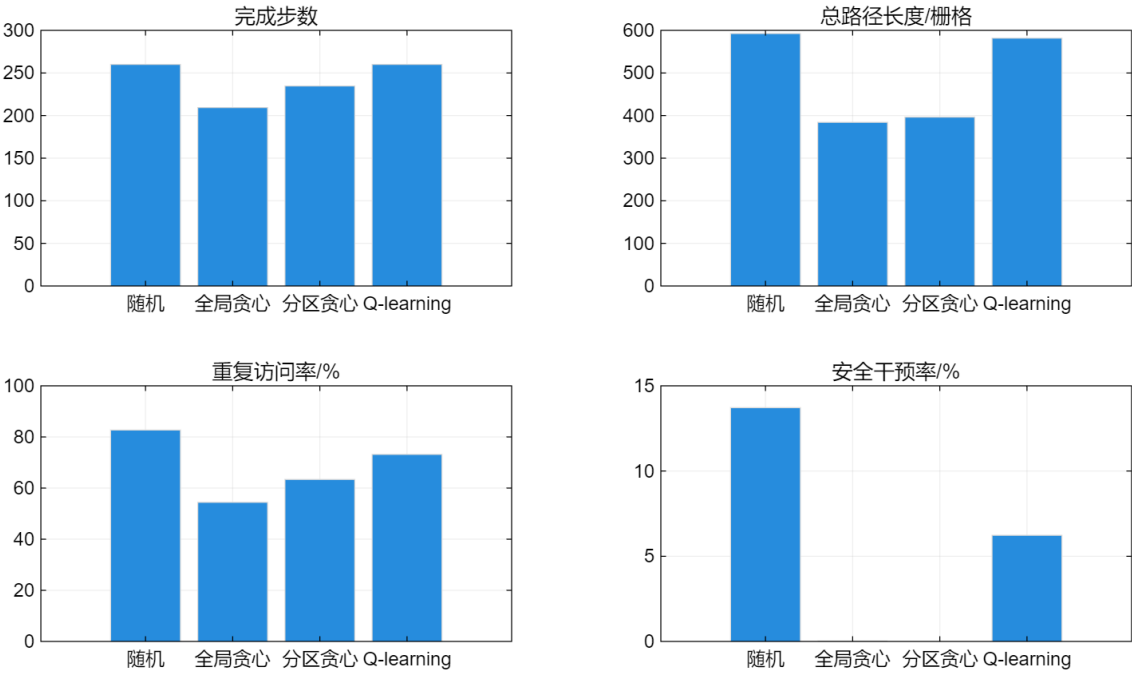


图 5 基准条件下完成步数、路径长度、重复访问率和安全干预率对比

当前参数下，全局贪心取得最高的平均覆盖率和 44% 的完成率，路径长度与重复率也低于另外两种主要方法，综合效率最好。分区贪心的覆盖率比全局贪心低 6.44 个百分点，但机间最小距离由 1.000 提高到 1.721，固定分工换来了更明显的空间分离。Q-learning 覆盖率为 56.25%，高于随机参照却低于两种贪心方法，重复率仍达 73.18%。三种方法没有在所有指标上给出同一排序，但在当前静态、全局地图已知的条件下，全局贪心最适合作为实机初期的默认任务策略。

同步冲突裁决保证最终位置不会重合，但程序仍保留“冲突尝试”与“安全干预率”。全局贪心平均产生 0.08 次冲突尝试，安全干预率约为 0.02%；分区贪心在本组评价中没有产生冲突尝试。Q-learning 平均提出 48.60 个需要裁决的候选动作，安全干预率为 6.23%。随机参照的安全干预率最高，为 13.72%。这些差异说明三种决策方法在进入硬安全层之前已有不同的冲突倾向，最终位置无重合不能替代对策略本身的检查。

基准比较之后，障碍密度和机群规模实验继续考察三种主要方法。评价仍使用相同的 25 个地图种子；需要重新学习的 Q-learning 配置均以 3 个训练种子分别训练 1200 回合，不沿用基准配置的 Q 表。图中的覆盖率误差线按 25 个地图级结果计算 95% 置信区间，Q-learning 仍先在每张地图内对训练种子取平均。完成率采用以 25 张地图为样本量的 Wilson 区间。这些区间用于描述地图间的波动，不代替正式的假设检验。障碍密度是固定墙体之外额外加入的随机障碍比例，因此 0 也不是空地图。

障碍密度与机群规模敏感性

表 4 不同附加障碍密度下的平均覆盖率 (%)

附加障碍密度	随机	全局贪心	分区贪心	Q-learning	Q-learning 重复率
0	48.94	96.14	96.14	76.87	62.32
0.03	46.66	91.53	88.99	48.17	79.54
0.05	46.82	88.96	82.52	56.25	73.18
0.08	43.56	75.11	67.65	35.44	85.26

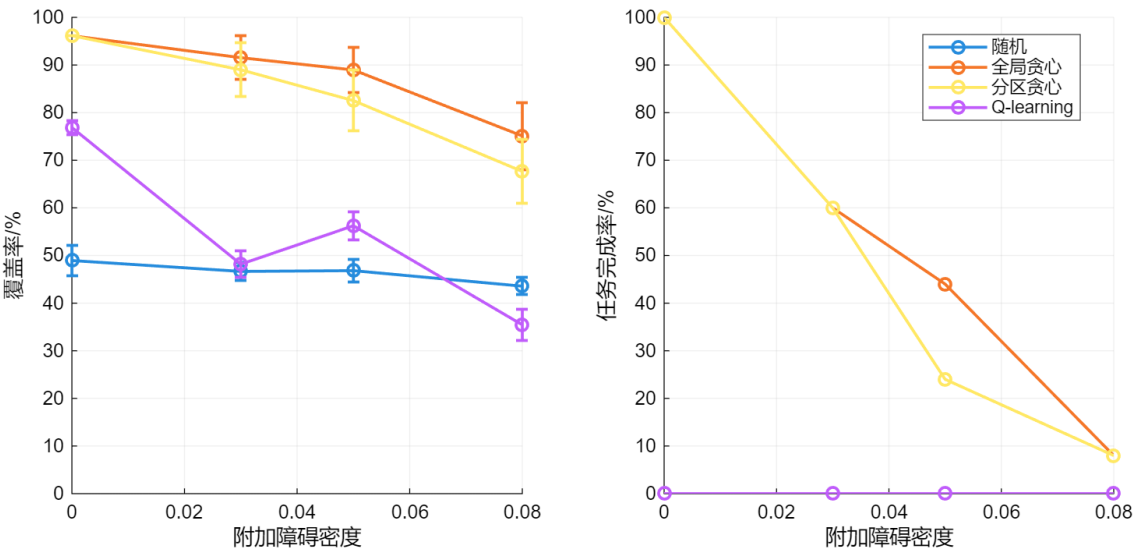


图 6 障碍密度变化对覆盖率和完成率的影响

障碍增加后，三种主要方法的覆盖率均有下降。附加密度由 0 提高到 0.08 时，全局贪心由 96.14% 降至 75.11%，分区贪心由 96.14% 降至 67.65%，两者仍保持前两位；Q-learning 由 76.87% 降至 35.44%，并低于同条件下的随机参照 43.56%，重复率同时由 62.32% 升至 85.26%。0.03 与 0.05 两档之间并非单调，训练随机性和具体拓扑都会影响结果。最高障碍密度下可达自由格仍占全部自由格的 99.73%，性能下降主要不是不可达格造成，而是窄通道、死胡同和分区连通性增加了决策难度。

表 5 不同无人机数量下的平均覆盖率 (%)

无人机数量	随机	全局贪心	分区贪心	Q-learning	Q-learning 重复率
2	34.71	76.24	73.79	40.53	77.71
3	46.82	88.96	82.52	56.25	73.18
4	61.35	91.88	90.47	63.10	79.23

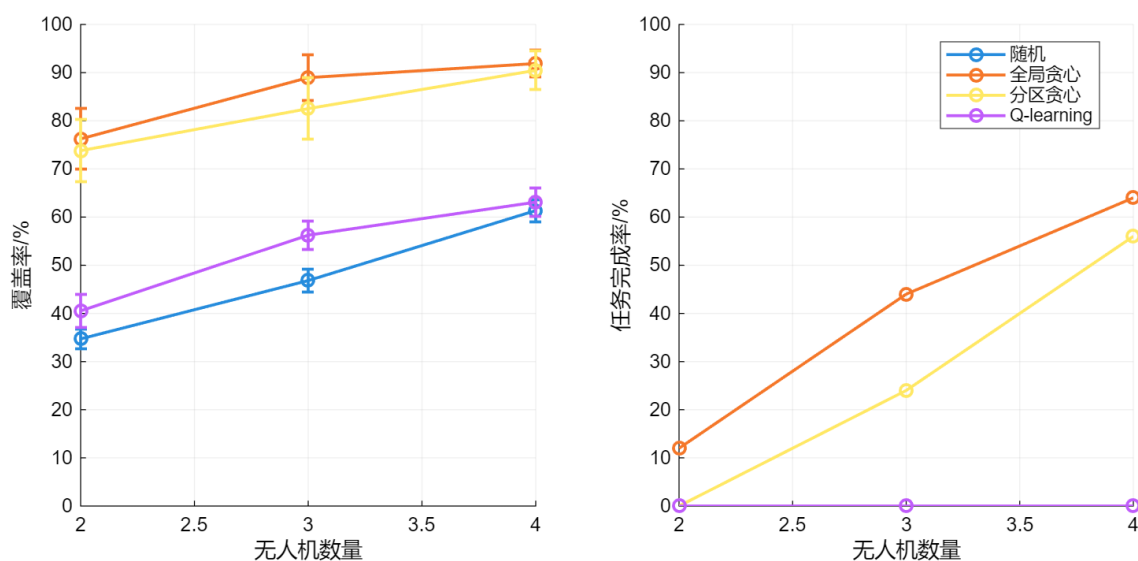


图 7 无人机数量变化对覆盖率和完成率的影响

增加无人机后, 三种主要方法的覆盖率都有提高。全局贪心由两机的 76.24% 提高到四机的 91.88%, 分区贪心由 73.79% 提高到 90.47%, 四机时两者已经接近; Q-learning 由 40.53% 提高到 63.10%, 但三个机数下完成率仍全部为 0, 四机重复率也有 79.23%。多机并行扩大了访问范围, 固定分区在四机条件下减少了相对损失, 而当前 Q-learning 仍没有解决短周期重复和尾部区域清理问题。

Q-learning 的进一步分析

基准结果中, Q-learning 的覆盖率明显低于两种贪心方法。只看最终指标还无法判断差距来自训练波动还是奖励设计, 因此下面继续检查训练曲线、奖励项和局部观测扰动。贪心方法没有训练过程和奖励函数, 对它们做同类消融没有实际含义。

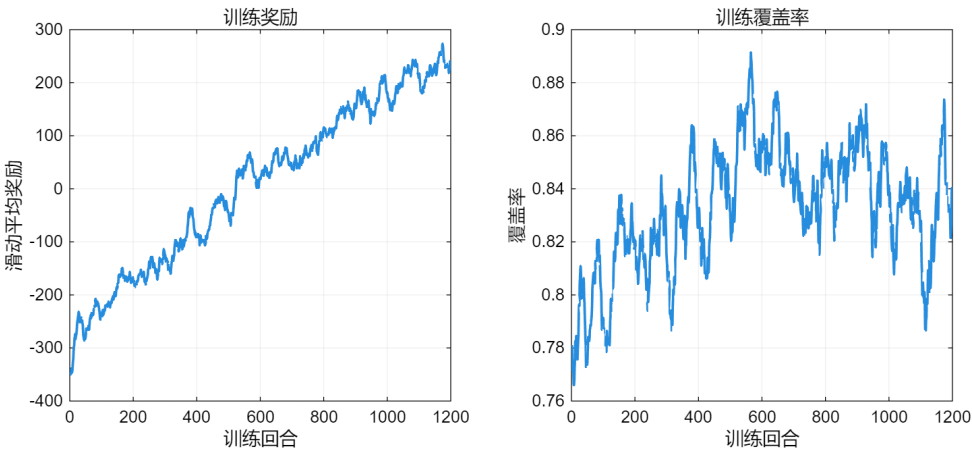


图 8 Q-learning 三个训练种子的平均训练曲线

训练过程与种子差异 三个训练种子的平均评价覆盖率分别为 55.53%、59.36% 和 53.85%，极差为 5.51 个百分点，地图级汇总均值为 56.25%。训练后期曲线趋于平缓，说明继续增加少量回合未必能明显改善当前结果；不同种子之间仍有差距，后续分析因此采用三张 Q 表的汇总值，不以表现最好的一次代表算法性能。

奖励函数消融 奖励消融保持地图、随机种子、训练回合和其余权重不变，分别删除重复访问惩罚、邻机距离惩罚、最近未访问距离引导和每步代价，每个版本均独立重新训练。这里比较的是 Q-learning 内部各奖励项的作用，不是把某个删减版本作为第四种主要方法。

表 6 Q-learning 奖励函数消融结果

配置	覆盖率/%	重复率/%	最小机间距离/栅格	冲突尝试	安全干预率/%
完整奖励	56.25	73.18	1.561	48.60	6.23
去重复惩罚	54.52	74.90	1.595	40.17	5.37
去邻机惩罚	44.04	80.54	1.884	26.67	3.42
去距离引导	34.19	85.81	3.517	14.23	1.82
去每步代价	44.49	79.70	1.863	31.93	4.10

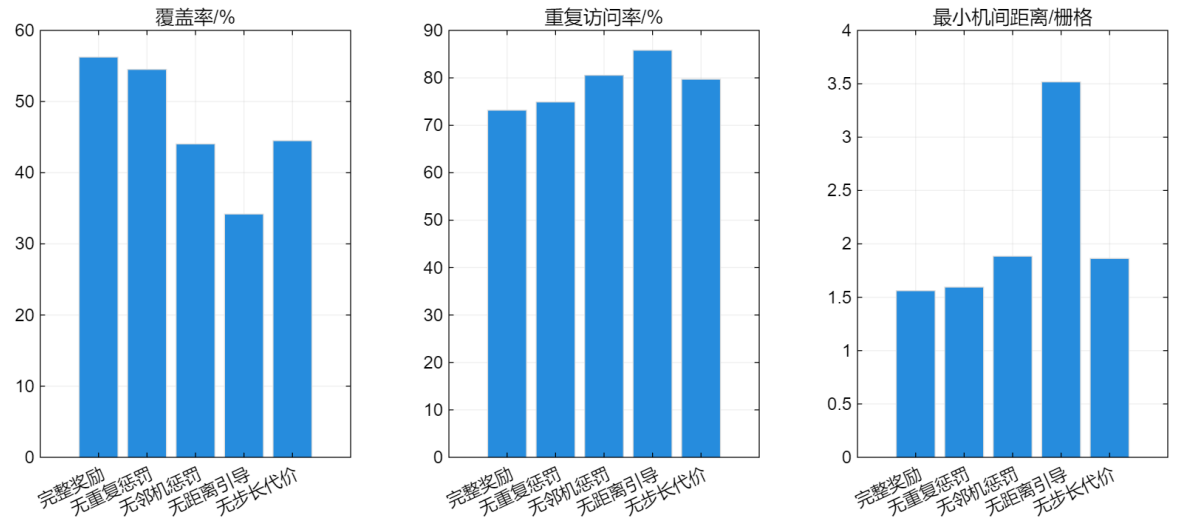


图 9 奖励函数消融对覆盖、重复搜索和机间距离的影响

距离引导项的贡献最稳定，删除后覆盖率下降 22.06 个百分点、重复率升至 85.81%。去掉邻机惩罚和每步代价后，覆盖率分别下降 12.21 和 11.76 个百分点。删除重复访问惩罚时，平均覆盖率只下降 1.73 个百分点，相比地图间波动幅度较小；本实验未设置配对假设检验，因此这里只报告差值，不把它解释为显著收益。低覆盖配置的最小机间距离反而较大、冲突尝试较少，主要是因为无人机活动范围缩小，并不代表策略更安全。奖励项的作用需要结合覆盖、移动量和安全干预共同判断。

局部观测延迟与丢包 最后一组实验沿用已训练的 Q 表，只在评价阶段为每架无人机维护带消息年龄的局部观测。延迟设为 0、2 和 5 步，丢包率设为 0、10% 和 30%。消息丢失后继续使用上一条已接收的邻机位置和覆盖地图，无人机仍知道自己的位置与自身访问记录。延迟和丢包只改变策略读取的信息，同步冲突裁决仍使用真实位置和完整障碍地图。严格来说，这是一组 Q-learning 局部观测扰动实验，而不是三种方法的通信性能比较，也不能代表端到端通信失效下的飞行安全。

表 7 局部观测扰动下 Q-learning 覆盖率与安全干预率

延迟/步	0% 丢包覆盖率	10% 丢包覆盖率	30% 丢包覆盖率	30% 丢包干预率
0	56.25	56.81	56.71	6.25
2	57.86	57.29	57.95	7.02
5	59.14	58.40	60.38	6.52

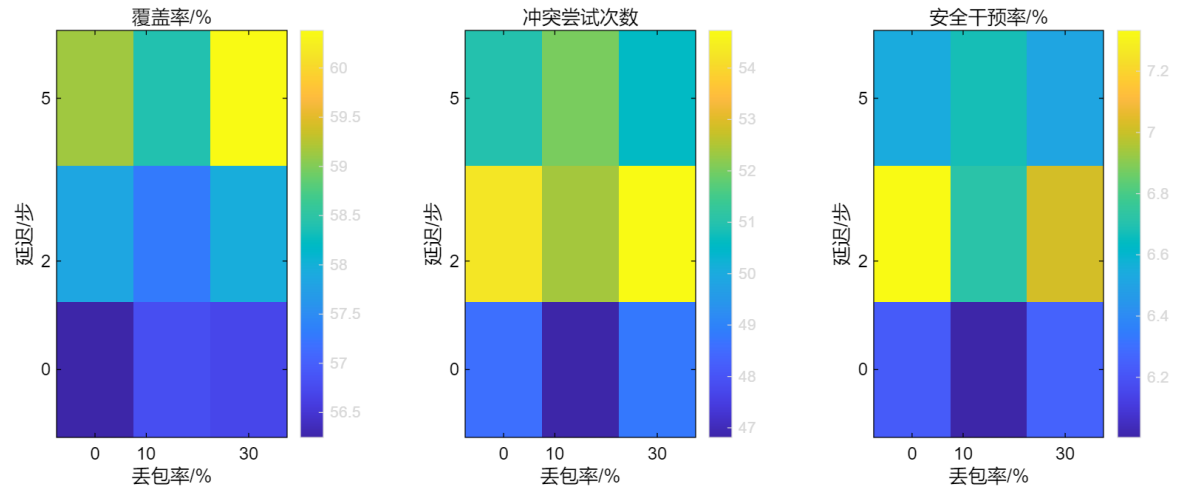


图 10 局部观测延迟和丢包对覆盖、冲突尝试及安全干预的影响

九组条件下的平均覆盖率为 56.25% 至 60.38%，随延迟或丢包增加没有出现单调变化；冲突尝试为 46.81 至 54.73 次，安全干预率为 6.00% 至 7.33%，变化方向同样不一致。图中区间用于展示地图间波动，本实验没有进行通信因素的正式假设检验。陈旧覆盖信息偶尔会打破局部短周期，可能是部分扰动条件均值略高的原因，但这不能被解释为“延迟有益”。此外，5 步延迟、30% 丢包条件下只有 1 个种子-地图组合达到 96% 覆盖，数量不足以支持完成率改善的判断。结合前述集中安全层的设定，这组结果至多说明当前决策器对所设观测扰动不敏感，不能外推到真实网络失效场景。

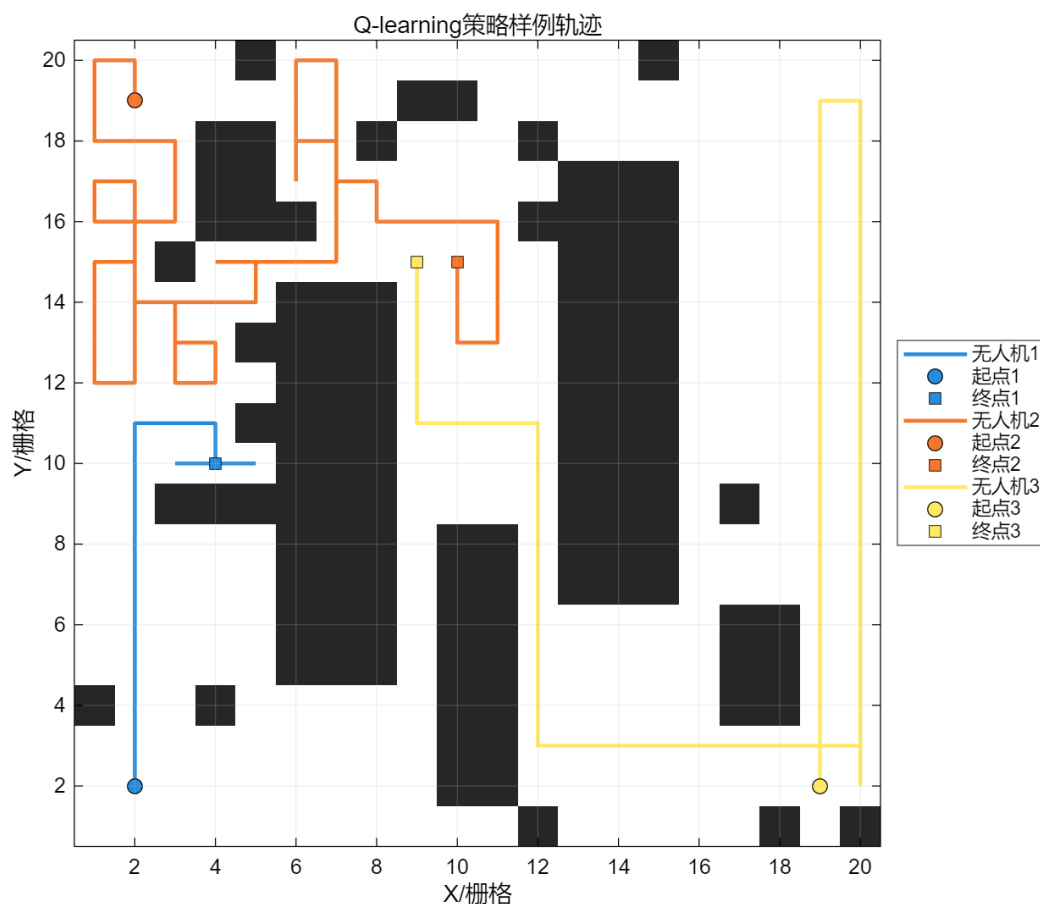


图 11 Q-learning 策略下的多机搜索轨迹样例（专项分析）

3.6 从仿真到实机的实施路线

实机验证应按风险逐级增加，不宜直接进行多机自主飞行。第一阶段是纯软件模型在环仿真，将 MATLAB 中的任务策略移植为 ROS 节点，并在 Gazebo 或其他物理仿真环境中接入 PX4 SITL，补入三维运动、传感器噪声、通信延迟和飞控动力学。此时要先确定任务目标的保持时间与到达容差，记录目标更新后触发的重规划次数，检查规划器是否出现连续取消、轨迹震荡或计算超时。验收指标还包括轨迹跟踪误差、最小障碍间距、通信中断后的停车响应和任务覆盖率。

当前 MATLAB 模型让所有无人机在同一个时刻选取动作并同步更新位置，尚未覆盖 ROS 节点异步运行的情况。SITL 测试中应给各机设置不同的任务决策频率，并注入计算抖动、消息延迟和到达顺序变化。安全检查不能再依赖集中式的同步真值，而要使用各机收到的带时间戳轨迹和本地感知结果；日志则记录消息年龄、决策耗时和最小实际间距，用来判断异步执行是否削弱安全裕量。

第二阶段为硬件在环与台架测试。在去除螺旋桨的条件下检查供电稳定性、MAVLink 通信、雷达点云帧率、CPU/GPU 负载、温度、时间同步和安全开关。通过 PX4 HIL 或回放实测 rosbag，检查规划节点在真实计算时延下是否会超时，以及定位

短时跳变时保护逻辑是否生效。

第三阶段才进入实飞。先完成手动姿态、定高和定点测试，再在防护网、低速、有系留绳或易于紧急接管的条件下完成单机航点飞行。单机避障稳定后，两机从开阔环境、分离高度开始验证轨迹交换，最后再增加机数和障碍复杂度。每一级测试都应事先设定通过条件与中止条件，一次正常起降不能代替对跟踪误差、规划时延和安全间距的验收。

为避免实验结论受单次环境影响，实机测试需要保留可重复的场景与统一的日志字段。可以使用可移动软质障碍物布置开阔场、窄通道和局部遮挡三类场景，每类场景分别运行若干次。日志中除了记录最终覆盖率，还应保存定位频率、轨迹规划耗时、跟踪误差、实际最小机间距离、延迟与丢包、重规划次数、人工接管次数和剩余电量。只有任务完成且全程未触发安全边界，才能记为一次有效完成。

3.7 可行性与风险分析

从原理上看，本方案是可行的。三种任务决策方法都只产生低频局部目标，不参与高频姿态稳定，其计算波动不会直接作用到电机。局部规划器在已建立的占据地图上生成动力学可行轨迹，PX4 使用状态反馈跟踪该轨迹；四旋翼连续轨迹的平滑性和动力学约束可按最小 snap 轨迹生成方法处理 [8]。感知、规划和控制各有自己的更新频率，通过带时间戳的位姿、地图和轨迹接口交换信息。上层方法共用相同接口，也便于在日志回放中复现和比较。

当前结果还不能作为实机安全结论。二维栅格动作忽略了动力学和感知误差，仿真中的“无碰撞”只是离散状态经过同步冲突检查后的结果。实际飞行还会受到定位误差、控制滞后和通信延迟影响，几何上不相交的轨迹也可能失去安全间隔。实机使用的障碍膨胀量和机间安全半径应由机体尺寸、定位不确定度、最大跟踪误差与制动距离共同确定，不能直接沿用仿真中的一格距离。异步执行是另一项未验证因素，现有局部观测扰动实验没有改变集中安全层的同步裁决方式。

实机中最先需要防范的是定位退化，因为错误位姿会同时污染地图和轨迹。程序可联合检查协方差、点云匹配残差与位姿跳变，而不是依赖单一阈值。地图数据还要携带时间戳；感知帧率异常或地图长时间未更新时，规划器应停止继续使用旧地图。规划节点的心跳由看门狗监测，邻机消息是否失效则交给各机本地超时器判断。上述异常最终进入同一个有限状态机，根据风险程度从正常搜索切换到降速、悬停、返航或降落。状态转移由确定性规则触发，故障期间暂停所有上层自主决策。

4 研究体会

本次建模最直接的体会是，多机协同问题的难点不在于让多个个体同时运行，而在于定义清楚什么是“协同得好”。如果只看覆盖率，策略可能通过更长路径和更多重复访问换取较高结果；如果将安全惩罚设置得过大，无人机又会过于保守。因此，覆盖、时间、能耗和风险必须同时评价，不能用单一指标概括系统性能。

三种方法的对比改变了我对算法复杂度的看法。在静态、规则清楚而且全局地图可用的条件下，全局贪心反而最有效；加入固定分区后，覆盖效率有所下降，但机间距离增大。**Q-learning** 能够产生基本搜索行为，却没有表现出预期的全局协作能力。这个结果并不否定学习方法，只说明方法选择必须结合信息条件和任务环境，不能先设定某一种算法必然更好。

把仿真策略接到实机上以后，任务决策正确并不等于系统已经具备闭环安全飞行能力。供电纹波、时间同步误差、外参偏差和定位漂移都可能使规划轨迹失去可执行性，计算或通信延迟还会让问题继续积累。实机测试必须能够记录异常并随时中止，软件在环、硬件在环和单机低速飞行也不能跨级进行。

5 研究收获

这次建模最主要的收获是学会用统一指标比较不同决策思路。全局贪心重视即时覆盖效率，分区贪心把任务分配加入动作选择，**Q-learning** 则尝试从局部状态中学习行为。三者放在同一地图与安全规则下运行后，各自的优势和代价才真正可比。对于**Q-learning**，状态维数和奖励权重仍是额外问题：信息不足时无法表达全局任务，信息过多又会使 Q 表难以充分访问。

程序实现带来的收获更直接。**MATLAB** 工程已拆成主入口和功能包，地图、训练、评价、环境更新、扩展实验与绘图各有对应文件；运行时可以显示位置、覆盖区域和轨迹，结束后则保存 CSV 与图像。三种方法的障碍密度和机群规模实验沿用同一套评价接口，**Q-learning** 的奖励消融与局部观测扰动也复用了相同的环境更新和绘图模块。这样调整某类实验时，不必同时改动底层仿真逻辑，统计口径也更容易保持一致。

把二维决策器放回整机链路后，各模块的职责也清楚了：三种上层方法都回答“搜索哪里”，局部规划器处理“怎样到达”，**PX4** 完成轨迹跟踪。根据仿真结果，实机初期以全局贪心作为默认策略，分区贪心用于检查任务分工，**Q-learning** 保留为可切换的对照方法。按照这一接口，本课题无需复制 **EGO-Planner** 已有的轨迹优化算法，而应把工作集中在多机任务决策、重规划触发和系统集成上。

6 建议

仿真部分已经完成三种方法的基准比较以及障碍密度、机群规模敏感性测试，Q-learning 另有奖励消融和局部观测扰动结果。这些数据已经能够支持当前条件下的方法选择，下一步不必继续增加同类参数扫描。硬件装机前需根据电机、ESC、机载计算机和传感器的峰值功耗重新核算电池放电能力、DC-DC 余量、线束压降与散热；动力线和传感器线分开布置，完成雷达-IMU、雷达-机体外参标定以及飞控、计算机和传感器的时间同步。

软件侧应保留确定性的安全状态机。定位协方差过大、地图超时、规划超时或邻机消息过期时，系统应按预设条件降速、悬停、返航或降落，并暂停当前任务决策。实机测试时，三种上层方法应共用地图输入、EGO-Planner 参数和安全阈值，日志统一记录覆盖率、安全干预、消息年龄、规划耗时与人工接管次数。只有保持下游条件一致，实机结果才能与本报告的仿真对比对应。

测试顺序建议固定为无桨台架、PX4 硬件在环、单机低速、双机开阔场和多机障碍场。每一级都设定通过条件与中止条件，前一级未通过时不进入下一级。局部规划可接入 EGO-Planner，但本课题的后续重点在于任务目标接口、重规划触发、安全裕量和实际计算时延，而非重复实现开源框架已有的轨迹优化功能。

参考文献

- [1] Xin Zhou, Zhepei Wang, Hongkai Ye, Chao Xu, and Fei Gao. EGO-Planner: An ESDF-free gradient-based local planner for quadrotors. *IEEE Robotics and Automation Letters*, 2021.
- [2] Wei Xu, Yixi Cai, Dongjiao He, Jiarong Lin, and Fu Zhang. FAST-LIO2: Fast direct LiDAR-inertial odometry. *IEEE Transactions on Robotics*, 38(4):2053–2073, 2022.
- [3] Xin Zhou, Jiangchao Zhu, Hongyu Zhou, Chao Xu, and Fei Gao. EGO-Swarm: A fully autonomous and decentralized quadrotor swarm system in cluttered environments. In *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*, 2021.
- [4] Lorenz Meier, Dominik Honegger, and Marc Pollefeys. PX4: A node-based multi-threaded open source robotics framework for deeply embedded platforms. In *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*, pages 6235–6240, 2015.

- [5] Boyu Zhou, Yichen Zhang, Xinyi Chen, and Shaojie Shen. FUEL: Fast UAV exploration using incremental frontier structure and hierarchical planning. *IEEE Robotics and Automation Letters*, 2021.
- [6] Christopher J. C. H. Watkins and Peter Dayan. Q-learning. *Machine Learning*, 8:279–292, 1992.
- [7] Ming Tan. Multi-agent reinforcement learning: Independent vs. cooperative agents. In *Proceedings of the Tenth International Conference on Machine Learning*, pages 330–337, 1993.
- [8] Daniel Mellinger and Vijay Kumar. Minimum snap trajectory generation and control for quadrotors. In *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*, 2011.

附录 1：源代码

以下为仿真实验实际使用的 MATLAB 完整源代码。代码按模块排列，包括主入口、配置与地图、学习与评价、环境更新、扩展实验和结果输出。各文件位于 `scripts` 与 `scripts/+multi_uav` 目录，运行入口为 `run_multi_uav_sim`。

在 MATLAB 中进入 `scripts` 目录后，执行 `run_multi_uav_sim` 可运行完整实验；执行 `run_multi_uav_sim(false)` 可关闭界面并在后台生成数据和图像。附录代码按以下顺序组织：

1. 主入口：给出配置、训练、评价、扩展实验和输出之间的调用顺序；
2. `defaultConfig.m` 与 `buildMap.m`：定义实验参数、起点、障碍物和覆盖率口径；
3. `trainQPolicy.m` 与 `evaluatePolicy.m`：实现训练过程与评价流程；
4. `chooseAction.m`、`encodeState.m` 和 `stepAgents.m`：实现动作决策、状态编码、冲突裁决和奖励计算。

A.1 主入口与配置

```

1 function run_multi_uav_sim(showVisualization, saveAnimation, runExtended, useCheckpoint)
2 %RUN_MULTI_UAV_SIM Entry point for the cooperative-search simulation.
3 % Usage:
4 %   run_multi_uav_sim
5 %   run_multi_uav_sim(true, true)
6 %   run_multi_uav_sim(false, false, false)
7 %   run_multi_uav_sim(false, false, true, true)
8
9 clc;
10 close all;
11 rng(7);
12
13 if nargin < 1, showVisualization = true; end
14 if nargin < 2, saveAnimation = false; end
15 if nargin < 3, runExtended = true; end
16 if nargin < 4, useCheckpoint = false; end
17
18 rootDir = fileparts(fileparts(mfilename('fullpath')));
19 figDir = fullfile(rootDir, 'figures');
20 if ~exist(figDir, 'dir'), mkdir(figDir); end
21
22 cfg = multi_uav.defaultConfig();
23 policies = {'random', 'greedy', 'partitioned', 'qlearn'};
24 checkpointFile = fullfile(rootDir, 'baseline_checkpoint.mat');
25
26 if useCheckpoint && exist(checkpointFile, 'file')

```

```

27     saved = load(checkpointFile, 'Qs', 'trainReward', 'trainCoverage', ...
28         'results', 'qSeedResults');
29     Qs = saved.Qs;
30     trainReward = saved.trainReward;
31     trainCoverage = saved.trainCoverage;
32     results = saved.results;
33     qSeedResults = saved.qSeedResults;
34     results.qlearn = multi_uav.aggregateEvaluations(qSeedResults);
35     fprintf('Loaded baseline checkpoint.\n');
36 else
37     fprintf('Training Q-learning with %d independent seeds...\n', numel(cfg.trainSeeds));
38     [Qs, trainReward, trainCoverage, qResult, qSeedResults] = ...
39         multi_uav.trainQEnsembleParallel(cfg);
40     results = struct();
41     for i = 1:numel(policies) - 1
42         fprintf('Evaluating %s policy...\n', policies{i});
43         results.(policies{i}) = ...
44             multi_uav.evaluatePolicy(cfg, Qs{i}, policies{i});
45     end
46     results.qlearn = qResult;
47     save(checkpointFile, 'Qs', 'trainReward', 'trainCoverage', 'results', ...
48         'qSeedResults', 'cfg', 'policies');
49 end
50
51 multi_uav.writeOutputs(rootDir, results, policies, qSeedResults, cfg.trainSeeds);
52 multi_uav.plotBaseline(figDir, trainReward, trainCoverage, results, ...
53     policies, cfg, showVisualization);
54
55 if runExtended
56     multi_uav.runExperiments(rootDir, figDir, cfg, Qs, policies, showVisualization);
57 end
58 if showVisualization
59     multi_uav.animateSearch(figDir, cfg, results.qlearn, saveAnimation);
60 end
61
62 fprintf('Simulation outputs written to: %s\n', rootDir);
63 if showVisualization
64     fprintf('Close the figure windows when you finish inspecting the results.\n');
65 end
66 end

```

Listing 1 主程序入口 run_multi_uav_sim.m

```

1 function cfg = defaultConfig(numAgents)
2 %DEFAULTCONFIG Create the reproducible simulation and learning configuration.
3 if nargin < 1
4     numAgents = 3;
5 end
6
7 cfg.N = 20;
8 cfg.numAgents = numAgents;
9 cfg.maxSteps = 260;
10 cfg.trainEpisodes = 1200;
11 cfg.trainSeeds = [1 2 3];

```

```

12 cfg.trainSeed = cfg.trainSeeds(1);
13 cfg.evalEpisodes = 25;
14 cfg.evalSeeds = 1001 + (0:cfg.evalEpisodes - 1);
15 cfg.actions = [0 -1; 0 1; -1 0; 1 0; 0 0];
16
17 cfg.alpha = 0.18;
18 cfg.gamma = 0.92;
19 cfg.epsilon0 = 0.55;
20 cfg.epsilonMin = 0.05;
21 cfg.obstacleRate = 0.05;
22
23 cfg.stepCost = 0.03;
24 cfg.newCoverageReward = 2.5;
25 cfg.repeatPenalty = 0.45;
26 cfg.hoverPenalty = 0.12;
27 cfg.obstaclePenalty = 8.0;
28 cfg.conflictPenalty = 5.0;
29 cfg.nearAgentPenalty = 1.2;
30 cfg.distanceGain = 0.12;
31 cfg.distanceCost = 0.003;
32
33 cfg.starts = makeStarts(cfg.N, cfg.numAgents);
34 cfg.ownerMap = makeOwnerMap(cfg);
35 end
36
37 function starts = makeStarts(N, numAgents)
38 corners = [2 2; 2 N - 1; N - 1 2; N - 1 N - 1];
39 if numAgents > size(corners, 1)
40     error('This experiment supports at most four agents.');
```

Listing 2 统一参数配置 defaultConfig.m

A.2 地图与空间辅助函数

```

1 function [obstacles, freeMask, reachableFraction] = buildMap(cfg)
```



```

2 % Build the obstacle map and compute the reachable free-space mask.
3 N = cfg.N;
4 obstacles = false(N, N);
5
6 % Fixed walls and blocks. Coordinates are [x, y].
7 obstacles(6:8, 5:14) = true;
8 obstacles(13:15, 7:17) = true;
9 obstacles(10:11, 2:8) = true;
10 obstacles(4:5, 16:18) = true;
11 obstacles(17:18, 4:6) = true;
12
13 % Add sparse random clutter on top of the fixed walls while keeping starts free.
14 numRandom = round(cfg.obstacleRate * N * N);
15 placed = 0;
16 while placed < numRandom
17     x = randi(N);
18     y = randi(N);
19     if ~obstacles(x, y) && ~isStart(cfg, [x y])
20         obstacles(x, y) = true;
21         placed = placed + 1;
22     end
23 end
24 for k = 1:size(cfg.starts, 1)
25     obstacles(cfg.starts(k, 1), cfg.starts(k, 2)) = false;
26 end
27 allFree = ~obstacles;
28 freeMask = reachableFromStarts(allFree, cfg.starts);
29 reachableFraction = nnz(freeMask) / nnz(allFree);
30 end
31
32 function reachable = reachableFromStarts(allFree, starts)
33 N = size(allFree, 1);
34 reachable = false(size(allFree));
35 queue = zeros(numel(allFree), 2);
36 head = 1;
37 tail = 0;
38 for i = 1:size(starts, 1)
39     p = starts(i, :);
40     if allFree(p(1), p(2)) && ~reachable(p(1), p(2))
41         tail = tail + 1;
42         queue(tail, :) = p;
43         reachable(p(1), p(2)) = true;
44     end
45 end
46 dirs = [-1 0; 1 0; 0 -1; 0 1];
47 while head <= tail
48     p = queue(head, :);
49     head = head + 1;
50     for d = 1:4
51         q = p + dirs(d, :);
52         if q(1) >= 1 && q(1) <= N && q(2) >= 1 && q(2) <= N && ...
53             allFree(q(1), q(2)) && ~reachable(q(1), q(2))
54             reachable(q(1), q(2)) = true;
55             tail = tail + 1;
56             queue(tail, :) = q;

```

```

57     end
58   end
59 end
60 end
61
62 function tf = isStart(cfg, p)
63 tf = any(all(cfg.starts == p, 2));
64 end

```

Listing 3 地图生成与可达区域计算 buildMap.m

```

1 function tf = isFree(p, obstacles, N)
2 % Test map bounds and obstacle occupancy.
3 tf = p(1) >= 1 && p(1) <= N && p(2) >= 1 && p(2) <= N && ~obstacles(p(1), p(2));
4 end

```

Listing 4 自由栅格判断 isFree.m

```

1 function target = nearestUnvisited(p, visited, freeMask)
2 % Find the nearest reachable unvisited cell.
3 mask = freeMask & ~visited;
4 [xs, ys] = find(mask);
5 if isempty(xs)
6     target = [];
7     return;
8 end
9 d = abs(xs - p(1)) + abs(ys - p(2));
10 [~, idx] = min(d);
11 target = [xs(idx), ys(idx)];
12 end

```

Listing 5 最近未访问栅格 nearestUnvisited.m

```

1 function target = nearestAssignedUnvisited(p, aidx, visited, freeMask, cfg)
2 % Find the nearest unvisited cell assigned to one agent.
3 mask = freeMask & ~visited;
4 [xs, ys] = find(mask);
5 if isempty(xs)
6     target = [];
7     return;
8 end
9
10 assigned = cfg.ownerMap(sub2ind([cfg.N cfg.N], xs, ys)) == aidx;
11 if ~any(assigned)
12     target = multi_uav.nearestUnvisited(p, visited, freeMask);
13     return;
14 end
15
16 assignedX = xs(assigned);
17 assignedY = ys(assigned);
18 d = abs(assignedX - p(1)) + abs(assignedY - p(2));
19 [~, idx] = min(d);

```

```

20 target = [assignedX(idx), assignedY(idx)];
21 end

```

Listing 6 分区目标搜索 nearestAssignedUnvisited.m

```

1 function d = nearestAgentDistance(pos, aidx)
2 % Measure separation from one agent to its nearest neighbor.
3 others = setdiff(1:size(pos, 1), aidx);
4 if isempty(others)
5     d = inf;
6     return;
7 end
8 d = min(vecnorm(pos(others, :) - pos(aidx, :), 2, 2));
9 end

```

Listing 7 最近邻机距离 nearestAgentDistance.m

```

1 function c = coverageRatio(visited, freeMask)
2 % Compute reachable-area coverage.
3 c = nnz(visited & freeMask) / nnz(freeMask);
4 end

```

Listing 8 覆盖率计算 coverageRatio.m

A.3 Q-learning 训练与评价

```

1 function [Q, epReward, epCoverage] = trainQPolicy(cfg)
2 % Train one shared Q table for a single random seed.
3 numStates = 2^13;
4 numActions = size(cfg.actions, 1);
5 Q = zeros(numStates, numActions);
6 epReward = zeros(cfg.trainEpisodes, 1);
7 epCoverage = zeros(cfg.trainEpisodes, 1);
8
9 for ep = 1:cfg.trainEpisodes
10     rng(10000 + 100000 * (cfg.trainSeed - 1) + ep);
11     [obstacles, freeMask] = multi_uav.buildMap(cfg);
12     eps = max(cfg.epsilonMin, cfg.epsilon0 * (1 - ep / cfg.trainEpisodes));
13     [pos, visited] = multi_uav.resetEpisode(cfg, freeMask);
14     totalReward = 0;
15     for t = 1:cfg.maxSteps
16         states = zeros(cfg.numAgents, 1);
17         actions = zeros(cfg.numAgents, 1);
18         for aidx = 1:cfg.numAgents
19             states(aidx) = multi_uav.encodeState(pos, aidx, visited, obstacles, freeMask, cfg);
20             if rand < eps
21                 actions(aidx) = randi(numActions);
22             else
23                 actions(aidx) = multi_uav.randomArgmax(Q(states(aidx), :));
24             end

```

```

25     end
26     [newPos, rewards, visited] = multi_uav.stepAgents( ...
27         pos, actions, visited, obstacles, freeMask, cfg);
28     for aidx = 1:cfg.numAgents
29         s2 = multi_uav.encodeState(newPos, aidx, visited, obstacles, freeMask, cfg);
30         s = states(aidx);
31         act = actions(aidx);
32         Q(s, act) = Q(s, act) + cfg.alpha * ( ...
33             rewards(aidx) + cfg.gamma * max(Q(s2, :)) - Q(s, act));
34     end
35     pos = newPos;
36     totalReward = totalReward + sum(rewards);
37     if multi_uav.coverageRatio(visited, freeMask) >= 0.96
38         break;
39     end
40 end
41 epReward(ep) = totalReward;
42 epCoverage(ep) = multi_uav.coverageRatio(visited, freeMask);
43 end
44 end

```

Listing 9 单种子 Q-learning 训练 trainQPolicy.m

```

1 function [Qs, meanReward, meanCoverage, combined, seedResults] = trainQEnsembleParallel(cfg)
2 % Train and evaluate the independent-seed ensemble in parallel.
3 numSeeds = numel(cfg.trainSeeds);
4 Qs = cell(numSeeds, 1);
5 rewards = zeros(cfg.trainEpisodes, numSeeds);
6 coverages = zeros(cfg.trainEpisodes, numSeeds);
7 seedResults = cell(numSeeds, 1);
8 parfor si = 1:numSeeds
9     seedCfg = cfg;
10    seedCfg.trainSeed = cfg.trainSeeds(si);
11    [Qs{si}, rewards(:, si), coverages(:, si)] = multi_uav.trainQPolicy(seedCfg);
12    seedResults{si} = multi_uav.evaluatePolicy(seedCfg, Qs{si}, 'qlearn');
13 end
14 meanReward = mean(rewards, 2);
15 meanCoverage = mean(coverages, 2);
16 combined = multi_uav.aggregateEvaluations(seedResults);
17 end

```

Listing 10 并行多种子训练 trainQEnsembleParallel.m

```

1 function [Qs, combined] = trainQEnsembleSequential(cfg)
2 % Train an ensemble sequentially for parameter studies.
3 numSeeds = numel(cfg.trainSeeds);
4 Qs = cell(numSeeds, 1);
5 seedResults = cell(numSeeds, 1);
6 for si = 1:numSeeds
7     seedCfg = cfg;
8     seedCfg.trainSeed = cfg.trainSeeds(si);
9     [Qs{si}, ~, ~] = multi_uav.trainQPolicy(seedCfg);
10    seedResults{si} = multi_uav.evaluatePolicy(seedCfg, Qs{si}, 'qlearn');

```

```

11 end
12 combined = multi_uav.aggregateEvaluations(seedResults);
13 end

```

Listing 11 顺序多种子训练 trainQEnsembleSequential.m

```

1 function result = evaluatePolicy(cfg, Q, policy, commDelay, commLoss)
2 % Run reproducible evaluation maps for one policy.
3 if nargin < 4
4     commDelay = 0;
5 end
6 if nargin < 5
7     commLoss = 0;
8 end
9 coverages = zeros(cfg.evalEpisodes, cfg.maxSteps);
10 pathLengths = zeros(cfg.evalEpisodes, 1);
11 collisionCounts = zeros(cfg.evalEpisodes, 1);
12 repeatRates = zeros(cfg.evalEpisodes, 1);
13 finishSteps = zeros(cfg.evalEpisodes, 1);
14 minDistances = zeros(cfg.evalEpisodes, 1);
15 safetyRates = zeros(cfg.evalEpisodes, 1);
16 reachableFractions = zeros(cfg.evalEpisodes, 1);
17 completed = false(cfg.evalEpisodes, 1);
18 sampleTraj = [];
19 sampleSteps = 0;
20 sampleObstacles = [];
21 sampleFreeMask = [];
22
23 for ep = 1:cfg.evalEpisodes
24     rng(cfg.evalSeeds(ep));
25     [obstacles, freeMask, reachableFractions(ep)] = multi_uav.buildMap(cfg);
26     [pos, visited] = multi_uav.resetEpisode(cfg, freeMask);
27     perceivedPos = repmat(pos, 1, 1, cfg.numAgents);
28     perceivedVisited = repmat(visited, 1, 1, cfg.numAgents);
29     localVisited = repmat(visited, 1, 1, cfg.numAgents);
30     posHistory = cell(cfg.maxSteps, 1);
31     visitedHistory = cell(cfg.maxSteps, 1);
32     traj = zeros(cfg.maxSteps + 1, cfg.numAgents, 2);
33     traj(1, :, :) = pos;
34     repeated = 0;
35     collisions = 0;
36     safetyInterventions = 0;
37     pathLen = 0;
38     minDist = inf;
39     for t = 1:cfg.maxSteps
40         oldPos = pos;
41         posHistory{t} = pos;
42         visitedHistory{t} = visited;
43         actions = zeros(cfg.numAgents, 1);
44         shielded = false(cfg.numAgents, 1);
45         for aidx = 1:cfg.numAgents
46             sensedPos = pos;
47             sensedVisited = visited;
48             if commDelay > 0 || commLoss > 0

```

```

49         sourceIdx = max(1, t - commDelay);
50         delayedPos = posHistory{sourceIdx};
51         delayedVisited = visitedHistory{sourceIdx};
52         for other = 1:cfg.numAgents
53             if other ~= aidx && packetReceived(commLoss)
54                 perceivedPos(other, :, aidx) = delayedPos(other, :);
55             end
56         end
57         if packetReceived(commLoss)
58             perceivedVisited(:, :, aidx) = delayedVisited;
59         end
60         sensedPos = perceivedPos(:, :, aidx);
61         sensedPos(aidx, :) = pos(aidx, :);
62         sensedVisited = perceivedVisited(:, :, aidx) | localVisited(:, :, aidx);
63     end
64     [actions(aidx), shielded(aidx)] = multi_uav.chooseAction( ...
65         policy, sensedPos, aidx, sensedVisited, obstacles, freeMask, cfg, Q);
66 end
67 [pos, ~, visited, conflicted, newlyCovered] = multi_uav.stepAgents( ...
68     pos, actions, visited, obstacles, freeMask, cfg);
69 collisions = collisions + sum(conflicted);
70 repeated = repeated + sum(~conflicted & ~newlyCovered);
71 safetyInterventions = safetyInterventions + sum(shielded | conflicted);
72 for i = 1:cfg.numAgents
73     pathLen = pathLen + norm(pos(i, :) - oldPos(i, :), 2);
74     localVisited(pos(i, 1), pos(i, 2), i) = true;
75 end
76 minDist = min(minDist, minAgentDistance(pos));
77 coverages(ep, t) = multi_uav.coverageRatio(visited, freeMask);
78 traj(t + 1, :, :) = pos;
79 if coverages(ep, t) >= 0.96
80     coverages(ep, t + 1:end) = coverages(ep, t);
81     break;
82 end
83 end
84 if ep == 1
85     sampleSteps = t;
86     sampleTraj = traj(1:t + 1, :, :);
87     sampleObstacles = obstacles;
88     sampleFreeMask = freeMask;
89 end
90 pathLengths(ep) = pathLen;
91 collisionCounts(ep) = collisions;
92 doneIdx = find(coverages(ep, :) >= 0.96, 1, 'first');
93 if isempty(doneIdx)
94     finishSteps(ep) = cfg.maxSteps;
95 else
96     finishSteps(ep) = doneIdx;
97     completed(ep) = true;
98 end
99 repeatRates(ep) = repeated / max(1, t * cfg.numAgents);
100 safetyRates(ep) = safetyInterventions / max(1, t * cfg.numAgents);
101 minDistances(ep) = minDist;
102 end
103

```

```

004 result.coverageCurve = mean(coverages, 1);
005 result.finalCoverage = mean(coverages(:, end));
006 result.finalCoverageStd = std(coverages(:, end));
007 result.pathLength = mean(pathLengths);
008 result.pathLengthStd = std(pathLengths);
009 result.collisions = mean(collisionCounts);
010 result.collisionsStd = std(collisionCounts);
011 result.repeatRate = mean(repeatRates);
012 result.repeatRateStd = std(repeatRates);
013 result.finishSteps = mean(finishSteps);
014 result.finishStepsStd = std(finishSteps);
015 result.minDistance = mean(minDistances);
016 result.minDistanceStd = std(minDistances);
017 result.completionRate = mean(completed);
018 result.safetyRate = mean(safetyRates);
019 result.safetyRateStd = std(safetyRates);
020 result.reachableFraction = mean(reachableFractions);
021 result.reachableFractionStd = std(reachableFractions);
022 result.finalCoverageCI95 = 1.96 * result.finalCoverageStd / sqrt(cfg.evalEpisodes);
023 result.completionRateCI95 = wilsonHalfWidth(result.completionRate, cfg.evalEpisodes);
024 result.coverageSamples = coverages;
025 result.finalCoverageSamples = coverages(:, end);
026 result.finishStepSamples = finishSteps;
027 result.pathLengthSamples = pathLengths;
028 result.collisionSamples = collisionCounts;
029 result.repeatRateSamples = repeatRates;
030 result.minDistanceSamples = minDistances;
031 result.safetyRateSamples = safetyRates;
032 result.reachableFractionSamples = reachableFractions;
033 result.completionSamples = completed;
034 result.sampleTraj = sampleTraj;
035 result.sampleSteps = sampleSteps;
036 result.sampleObstacles = sampleObstacles;
037 result.sampleFreeMask = sampleFreeMask;
038 end
039
040 function d = minAgentDistance(pos)
041     d = inf;
042     for i = 1:size(pos, 1)
043         for j = i + 1:size(pos, 1)
044             d = min(d, norm(pos(i, :) - pos(j, :), 2));
045         end
046     end
047 end
048
049 function received = packetReceived(lossRate)
050     received = lossRate <= 0 || rand >= lossRate;
051 end
052
053 function halfWidth = wilsonHalfWidth(p, n)
054     z = 1.96;
055     denominator = 1 + z^2 / n;
056     halfWidth = z / denominator * sqrt(p * (1 - p) / n + z^2 / (4 * n^2));
057 end

```

Listing 12 单策略评价 evaluatePolicy.m

```

1 function combined = evaluateQEnsemble(cfg, Qs, commDelay, commLoss)
2 % Evaluate all learned tables and aggregate their samples.
3 if nargin < 3
4     commDelay = 0;
5 end
6 if nargin < 4
7     commLoss = 0;
8 end
9 seedResults = cell(numel(Qs), 1);
10 for si = 1:numel(Qs)
11     seedResults{si} = multi_uav.evaluatePolicy(cfg, Qs{si}, 'qlearn', commDelay, commLoss);
12 end
13 combined = multi_uav.aggregateEvaluations(seedResults);
14 end

```

Listing 13 Q 表集合评价 evaluateQEnsemble.m

```

1 function result = aggregateEvaluations(seedResults)
2 % Average training-seed results on each map before estimating uncertainty.
3 result = seedResults{1};
4 numSeeds = numel(seedResults);
5 numMaps = size(result.coverageSamples, 1);
6 numSteps = size(result.coverageSamples, 2);
7 coverageBySeed = zeros(numMaps, numSteps, numSeeds);
8 metricNames = {'finishStepSamples', 'pathLengthSamples', 'collisionSamples', ...
9     'repeatRateSamples', 'minDistanceSamples', 'safetyRateSamples', ...
10    'reachableFractionSamples', 'completionSamples'};
11 metricData = struct();
12 for mi = 1:numel(metricNames)
13     metricData.(metricNames{mi}) = zeros(numMaps, numSteps);
14 end
15 for si = 1:numSeeds
16     r = seedResults{si};
17     if size(r.coverageSamples, 1) ~= numMaps || ...
18         size(r.coverageSamples, 2) ~= numSteps
19         error('All training seeds must use the same evaluation maps and horizon.');

```



```

33 safetyRates = mean(metricData.safetyRateSamples, 2);
34 reachableFractions = mean(metricData.reachableFractionSamples, 2);
35 completed = mean(metricData.completionSamples, 2);
36 numSamples = numMaps;
37 result.coverageCurve = mean(coverageSamples, 1);
38 result.finalCoverage = mean(coverageSamples(:, end));
39 result.finalCoverageStd = std(coverageSamples(:, end));
40 result.finalCoverageCI95 = 1.96 * result.finalCoverageStd / sqrt(numSamples);
41 result.finishSteps = mean(finishSteps);
42 result.finishStepsStd = std(finishSteps);
43 result.pathLength = mean(pathLengths);
44 result.pathLengthStd = std(pathLengths);
45 result.collisions = mean(collisionCounts);
46 result.collisionsStd = std(collisionCounts);
47 result.repeatRate = mean(repeatRates);
48 result.repeatRateStd = std(repeatRates);
49 result.minDistance = mean(minDistances);
50 result.minDistanceStd = std(minDistances);
51 result.safetyRate = mean(safetyRates);
52 result.safetyRateStd = std(safetyRates);
53 result.reachableFraction = mean(reachableFractions);
54 result.reachableFractionStd = std(reachableFractions);
55 result.completionRate = mean(completed);
56 result.completionRateCI95 = wilsonHalfWidth(result.completionRate, numSamples);
57 result.coverageSamples = coverageSamples;
58 result.finalCoverageSamples = coverageSamples(:, end);
59 result.finishStepSamples = finishSteps;
60 result.pathLengthSamples = pathLengths;
61 result.collisionSamples = collisionCounts;
62 result.repeatRateSamples = repeatRates;
63 result.minDistanceSamples = minDistances;
64 result.safetyRateSamples = safetyRates;
65 result.reachableFractionSamples = reachableFractions;
66 result.completionSamples = completed;
67 end
68
69 function halfWidth = wilsonHalfWidth(p, n)
70 z = 1.96;
71 denominator = 1 + z^2 / n;
72 halfWidth = z / denominator * sqrt(p * (1 - p) / n + z^2 / (4 * n^2));
73 end

```

Listing 14 多种子统计汇总 aggregateEvaluations.m

A.4 动作选择与环境更新

```

1 function [pos, visited] = resetEpisode(cfg, freeMask)
2 % Initialize agent positions and the visited mask.
3 pos = cfg.starts;
4 visited = false(cfg.N, cfg.N);
5 for k = 1:cfg.numAgents
6     visited(pos(k, 1), pos(k, 2)) = true;

```

```

7 end
8 visited(~freeMask) = false;
9 end

```

Listing 15 回合初始化 resetEpisode.m

```

1 function [act, shielded] = chooseAction(policy, pos, aidx, visited, obstacles, freeMask, cfg, Q)
2 % Select a safe candidate action for random, greedy, partitioned or learned policies.
3 shielded = false;
4 valid = validActions(pos, aidx, obstacles, cfg);
5 switch policy
6     case 'random'
7         rawAct = randi(size(cfg.actions, 1));
8         if ismember(rawAct, valid)
9             act = rawAct;
10        else
11            act = valid(randi(numel(valid)));
12            shielded = true;
13        end
14    case 'greedy'
15        act = greedyAction(pos, aidx, visited, obstacles, freeMask, cfg);
16    case 'partitioned'
17        act = partitionedGreedyAction(pos, aidx, visited, obstacles, freeMask, cfg);
18    case 'qlearn'
19        s = multi_uav.encodeState(pos, aidx, visited, obstacles, freeMask, cfg);
20        q = Q(s, :);
21        rawAct = multi_uav.randomArgmax(q);
22        if ismember(rawAct, valid)
23            act = rawAct;
24        else
25            q(setdiff(1:size(cfg.actions, 1), valid)) = -inf;
26            act = multi_uav.randomArgmax(q);
27            shielded = true;
28        end
29    otherwise
30        error('Unknown policy: %s', policy);
31 end
32 end
33
34 function valid = validActions(pos, aidx, obstacles, cfg)
35 valid = [];
36 for a = 1:size(cfg.actions, 1)
37     p = pos(aidx, :) + cfg.actions(a, :);
38     if multi_uav.isFree(p, obstacles, cfg.N)
39         valid(end + 1) = a; %#ok<AGROW>
40     end
41 end
42 if isempty(valid)
43     valid = size(cfg.actions, 1);
44 end
45 end
46
47 function act = greedyAction(pos, aidx, visited, obstacles, freeMask, cfg)
48 bestScore = -inf;

```

```

49 act = size(cfg.actions, 1);
50 for a = validActions(pos, aidx, obstacles, cfg)
51     p = pos(aidx, :) + cfg.actions(a, :);
52     score = 0;
53     if ~visited(p(1), p(2))
54         score = score + 3;
55     end
56     target = multi_uav.nearestUnvisited(p, visited, freeMask);
57     if ~isempty(target)
58         score = score - 0.08 * norm(target - p, 1);
59     end
60     others = pos;
61     others(aidx, :) = [];
62     for j = 1:size(others, 1)
63         if norm(p - others(j, :), 1) <= 1
64             score = score - 2.0;
65         end
66     end
67     if score > bestScore
68         bestScore = score;
69         act = a;
70     end
71 end
72 end
73
74 function act = partitionedGreedyAction(pos, aidx, visited, obstacles, freeMask, cfg)
75 bestScore = -inf;
76 act = size(cfg.actions, 1);
77 for a = validActions(pos, aidx, obstacles, cfg)
78     p = pos(aidx, :) + cfg.actions(a, :);
79     score = 0;
80     if ~visited(p(1), p(2))
81         score = score + 3;
82     end
83     target = multi_uav.nearestAssignedUnvisited(p, aidx, visited, freeMask, cfg);
84     if ~isempty(target)
85         score = score - 0.08 * norm(target - p, 1);
86     end
87     others = pos;
88     others(aidx, :) = [];
89     for j = 1:size(others, 1)
90         if norm(p - others(j, :), 1) <= 1
91             score = score - 2.0;
92         end
93     end
94     if score > bestScore
95         bestScore = score;
96         act = a;
97     end
98 end
99 end

```

Listing 16 四类策略的动作选择 chooseAction.m

```

1 function s = encodeState(pos, aidx, visited, obstacles, freeMask, cfg)
2 % Encode the 13-bit local observation as a Q-table row index.
3 p = pos(aidx, :);
4 bits = zeros(1, 13);
5 dirs = cfg.actions(1:4, :);
6 for i = 1:4
7     q = p + dirs(i, :);
8     bits(i) = ~multi_uav.isFree(q, obstacles, cfg.N);
9     if multi_uav.isFree(q, obstacles, cfg.N) && ~visited(q(1), q(2))
10         bits(4 + i) = 1;
11     end
12 end
13 bits(9) = multi_uav.nearestAgentDistance(pos, aidx) <= 2;
14
15 target = multi_uav.nearestAssignedUnvisited(p, aidx, visited, freeMask, cfg);
16 if ~isempty(target)
17     delta = target - p;
18     if abs(delta(2)) >= abs(delta(1))
19         if delta(2) < 0
20             bits(10) = 1; % north/up
21         else
22             bits(11) = 1; % south/down
23         end
24     else
25         if delta(1) < 0
26             bits(12) = 1; % west/left
27         else
28             bits(13) = 1; % east/right
29         end
30     end
31 end
32
33 s = 1;
34 for i = 1:numel(bits)
35     s = s + bits(i) * 2^(i - 1);
36 end
37 s = max(1, min(2^13, s));
38 end

```

Listing 17 13 位状态编码 encodeState.m

```

1 function [newPos, rewards, visited, conflicted, newlyCovered] = stepAgents( ...
2     pos, actions, visited, obstacles, freeMask, cfg)
3 % Resolve synchronous conflicts, move all agents and assign rewards.
4 numAgents = cfg.numAgents;
5 desired = pos + cfg.actions(actions, :);
6 rewards = -cfg.stepCost * ones(numAgents, 1);
7 conflicted = false(numAgents, 1);
8 newlyCovered = false(numAgents, 1);
9 staticInvalid = false(numAgents, 1);
10
11 for i = 1:numAgents
12     if ~multi_uav.isFree(desired(i, :), obstacles, cfg.N)

```

```

13     desired(i, :) = pos(i, :);
14     staticInvalid(i) = true;
15     conflicted(i) = true;
16     rewards(i) = rewards(i) - cfg.obstaclePenalty;
17 end
18 end
19
20 % Reject head-on swaps because both agents traverse the same edge.
21 for i = 1:numAgents
22     for j = i + 1:numAgents
23         if all(desired(i, :) == pos(j, :)) && all(desired(j, :) == pos(i, :)) && ...
24             any(desired(i, :) ~= pos(i, :))
25             desired(i, :) = pos(i, :);
26             desired(j, :) = pos(j, :);
27             conflicted([i j]) = true;
28         end
29     end
30 end
31
32 % Resolve multiple agents requesting the same vertex with rotating random priority.
33 order = randperm(numAgents);
34 for oi = 1:numAgents
35     i = order(oi);
36     if all(desired(i, :) == pos(i, :))
37         continue;
38     end
39     same = find(all(desired == desired(i, :), 2));
40     movers = same(any(desired(same, :) ~= pos(same, :), 2));
41     if numel(movers) > 1
42         winner = order(find(ismember(order, movers), 1, 'first'));
43         losers = setdiff(movers, winner);
44         desired(losers, :) = pos(losers, :);
45         conflicted(losers) = true;
46     end
47 end
48
49 % If an agent stays, no other agent may move into its occupied cell.
50 changed = true;
51 while changed
52     changed = false;
53     for i = 1:numAgents
54         if all(desired(i, :) == pos(i, :))
55             continue;
56         end
57         occupant = find(all(pos == desired(i, :), 2), 1);
58         if ~isempty(occupant) && all(desired(occupant, :) == pos(occupant, :))
59             desired(i, :) = pos(i, :);
60             conflicted(i) = true;
61             changed = true;
62         end
63     end
64 end
65
66 rewards(conflicted & ~staticInvalid) = rewards(conflicted & ~staticInvalid) - cfg.conflictPenalty
67 ;

```

```

67 newPos = desired;
68 oldVisited = visited;
69 for i = 1:numAgents
70     p = newPos(i, :);
71     if ~oldVisited(p(1), p(2))
72         newlyCovered(i) = true;
73         rewards(i) = rewards(i) + cfg.newCoverageReward;
74     else
75         rewards(i) = rewards(i) - cfg.repeatPenalty;
76     end
77     if actions(i) == size(cfg.actions, 1)
78         rewards(i) = rewards(i) - cfg.hoverPenalty;
79     end
80     visited(p(1), p(2)) = true;
81 end
82
83 for i = 1:numAgents
84     targetBefore = multi_uav.nearestAssignedUnvisited(pos(i, :), i, oldVisited, freeMask, cfg);
85     targetAfter = multi_uav.nearestAssignedUnvisited(newPos(i, :), i, visited, freeMask, cfg);
86     if ~isempty(targetBefore) && ~isempty(targetAfter)
87         distBefore = norm(targetBefore - pos(i, :), 1);
88         distAfter = norm(targetAfter - newPos(i, :), 1);
89         rewards(i) = rewards(i) + cfg.distanceGain * (distBefore - distAfter) ...
90             - cfg.distanceCost * distAfter;
91     end
92     if multi_uav.nearestAgentDistance(newPos, i) <= 1
93         rewards(i) = rewards(i) - cfg.nearAgentPenalty;
94     end
95 end
96 end

```

Listing 18 同步冲突裁决与奖励更新 stepAgents.m

```

1 function idx = randomArgmax(values)
2 % Break equal-value action ties without directional bias.
3 best = max(values);
4 candidates = find(values == best);
5 idx = candidates(randi(numel(candidates)));
6 end

```

Listing 19 随机并列最大值 randomArgmax.m

A.5 扩展实验与结果输出

```

1 function runExperiments(rootDir, figDir, cfg, Qs, policies, showVisualization)
2 %RUNEXPERIMENTS Execute the four extended parameter and robustness studies.
3 fprintf('Running obstacle-density sensitivity experiment...\n');
4 runObstacleSensitivity(rootDir, figDir, cfg, Qs, policies, showVisualization);
5 fprintf('Running fleet-size sensitivity experiment...\n');
6 runFleetSensitivity(rootDir, figDir, cfg, Qs, policies, showVisualization);
7 fprintf('Running reward ablation experiment...\n');

```

```

8 runRewardAblation(rootDir, figDir, cfg, Qs, showVisualization);
9 fprintf('Running communication robustness experiment...\n');
10 runCommunicationExperiment(rootDir, figDir, cfg, Qs, showVisualization);
11 end
12
13 function runObstacleSensitivity(rootDir, figDir, baseCfg, baseQs, policies, showVisualization)
14 rates = [0, 0.03, 0.05, 0.08];
15 coverage = zeros(numel(rates), numel(policies));
16 coverageCI = zeros(size(coverage));
17 completion = zeros(size(coverage));
18 allResults = cell(numel(rates), numel(policies));
19 parfor ri = 1:numel(rates)
20     cfg = baseCfg;
21     cfg.obstacleRate = rates(ri);
22     if abs(rates(ri) - baseCfg.obstacleRate) < 1e-12
23         qResult = multi_uav.evaluateQEnsemble(cfg, baseQs);
24     else
25         [~, qResult] = multi_uav.trainQEnsembleSequential(cfg);
26     end
27     row = cell(1, numel(policies));
28     for pi = 1:numel(policies) - 1
29         row{pi} = multi_uav.evaluatePolicy(cfg, baseQs{1}, policies{pi});
30     end
31     row{end} = qResult;
32     allResults(ri, :) = row;
33 end
34 fid = fopen(fullfile(rootDir, 'obstacle_sensitivity.csv'), 'w');
35 writeMetricHeader(fid, 'obstacle_rate,policy');
36 for ri = 1:numel(rates)
37     for pi = 1:numel(policies)
38         r = allResults{ri, pi};
39         fprintf(fid, '%.2f,%s,', rates(ri), policies{pi});
40         writeMetricValues(fid, r);
41         coverage(ri, pi) = 100 * r.finalCoverage;
42         coverageCI(ri, pi) = 100 * r.finalCoverageCI95;
43         completion(ri, pi) = 100 * r.completionRate;
44     end
45 end
46 fclose(fid);
47 plotSensitivity(figDir, rates, coverage, coverageCI, completion, policies, ...
48     '附加障碍密度', 'obstacle_sensitivity.png', showVisualization);
49 end
50
51 function runFleetSensitivity(rootDir, figDir, baseCfg, baseQs, policies, showVisualization)
52 counts = [2, 3, 4];
53 coverage = zeros(numel(counts), numel(policies));
54 coverageCI = zeros(size(coverage));
55 completion = zeros(size(coverage));
56 allResults = cell(numel(counts), numel(policies));
57 cfgList = cell(numel(counts), 1);
58 for ci = 1:numel(counts)
59     cfgList{ci} = baseCfg;
60     cfgList{ci}.numAgents = counts(ci);
61     agentCfg = multi_uav.defaultConfig(counts(ci));
62     cfgList{ci}.starts = agentCfg.starts;

```

```

63     cfgList{ci}.ownerMap = agentCfg.ownerMap;
64 end
65 parfor ci = 1:numel(counts)
66     cfg = cfgList{ci};
67     if counts(ci) == baseCfg.numAgents
68         qResult = multi_uav.evaluateQEnsemble(cfg, baseQs);
69     else
70         [~, qResult] = multi_uav.trainQEnsembleSequential(cfg);
71     end
72     row = cell(1, numel(policies));
73     for pi = 1:numel(policies) - 1
74         row{pi} = multi_uav.evaluatePolicy(cfg, baseQs{1}, policies{pi});
75     end
76     row{end} = qResult;
77     allResults(ci, :) = row;
78 end
79 fid = fopen(fullfile(rootDir, 'fleet_sensitivity.csv'), 'w');
80 writeMetricHeader(fid, 'num_agents,policy');
81 for ci = 1:numel(counts)
82     for pi = 1:numel(policies)
83         r = allResults{ci, pi};
84         fprintf(fid, '%d,%s,', counts(ci), policies{pi});
85         writeMetricValues(fid, r);
86         coverage(ci, pi) = 100 * r.finalCoverage;
87         coverageCI(ci, pi) = 100 * r.finalCoverageCI95;
88         completion(ci, pi) = 100 * r.completionRate;
89     end
90 end
91 fclose(fid);
92 plotSensitivity(figDir, counts, coverage, coverageCI, completion, policies, ...
93     '无人机数量', 'fleet_sensitivity.png', showVisualization);
94 end
95
96 function runRewardAblation(rootDir, figDir, baseCfg, baseQs, showVisualization)
97 labels = {'full', 'no_repeat', 'no_near', 'no_distance', 'no_step'};
98 coverage = zeros(numel(labels), 1);
99 repeat = zeros(size(coverage));
100 minDistance = zeros(size(coverage));
101 completion = zeros(size(coverage));
102 allResults = cell(numel(labels), 1);
103 parfor ai = 1:numel(labels)
104     cfg = baseCfg;
105     switch labels{ai}
106     case 'full'
107         qResult = multi_uav.evaluateQEnsemble(cfg, baseQs);
108     case 'no_repeat'
109         cfg.repeatPenalty = 0;
110         [~, qResult] = multi_uav.trainQEnsembleSequential(cfg);
111     case 'no_near'
112         cfg.nearAgentPenalty = 0;
113         [~, qResult] = multi_uav.trainQEnsembleSequential(cfg);
114     case 'no_distance'
115         cfg.distanceGain = 0;
116         cfg.distanceCost = 0;
117         [~, qResult] = multi_uav.trainQEnsembleSequential(cfg);

```



```

18         otherwise
19             cfg.stepCost = 0;
20             [~, qResult] = multi_uav.trainQEnsembleSequential(cfg);
21         end
22         allResults{ai} = qResult;
23     end
24     fid = fopen(fullfile(rootDir, 'reward_ablation.csv'), 'w');
25     writeMetricHeader(fid, 'ablation');
26     for ai = 1:numel(labels)
27         r = allResults{ai};
28         fprintf(fid, '%s,', labels{ai});
29         writeMetricValues(fid, r);
30         coverage(ai) = 100 * r.finalCoverage;
31         completion(ai) = 100 * r.completionRate;
32         repeat(ai) = 100 * r.repeatRate;
33         minDistance(ai) = r.minDistance;
34     end
35     fclose(fid);
36
37     f = figure('Visible', visibilityValue(showVisualization), 'Color', 'w', ...
38         'Position', [100 100 1050 380], 'Name', '奖励消融', 'NumberTitle', 'off');
39     values = {coverage, repeat, minDistance};
40     titles = {'覆盖率/%', '重复访问率/%', '最小机间距离/栅格'};
41     for k = 1:3
42         subplot(1, 3, k);
43         bar(values{k}); grid on;
44         set(gca, 'XTickLabel', {'完整奖励', '无重复惩罚', '无邻机惩罚', '无距离引导', '无步长代价'}, ...
45             'XTickLabelRotation', 25);
46         title(titles{k}); styleAxes(gca);
47     end
48     exportgraphics(f, fullfile(figDir, 'reward_ablation.png'), 'Resolution', 180);
49     closeIfHidden(f, showVisualization);
50 end
51
52 function runCommunicationExperiment(rootDir, figDir, cfg, Qs, showVisualization)
53     delays = [0, 2, 5];
54     losses = [0, 0.1, 0.3];
55     coverage = zeros(numel(delays), numel(losses));
56     safety = zeros(size(coverage));
57     conflicts = zeros(size(coverage));
58     fid = fopen(fullfile(rootDir, 'communication_robustness.csv'), 'w');
59     writeMetricHeader(fid, 'delay_steps,packet_loss');
60     for di = 1:numel(delays)
61         for li = 1:numel(losses)
62             r = multi_uav.evaluateQEnsemble(cfg, Qs, delays(di), losses(li));
63             fprintf(fid, '%d,%.1f,', delays(di), losses(li));
64             writeMetricValues(fid, r);
65             coverage(di, li) = 100 * r.finalCoverage;
66             safety(di, li) = 100 * r.safetyRate;
67             conflicts(di, li) = r.collisions;
68         end
69     end
70     fclose(fid);
71
72     f = figure('Visible', visibilityValue(showVisualization), 'Color', 'w', ...

```

```

173     'Position', [100 100 1050 360], 'Name', '通信异常影响', 'NumberTitle', 'off');
174     values = {coverage, conflicts, safety};
175     titles = {'覆盖率/%', '冲突尝试次数', '安全干预率/%'};
176     for k = 1:3
177         subplot(1, 3, k);
178         imagesc(losses * 100, delays, values{k}); colorbar;
179         xlabel('丢包率/%'); ylabel('延迟/步'); title(titles{k});
180         set(gca, 'YDir', 'normal', 'XTick', losses * 100, 'YTick', delays);
181         styleAxes(gca);
182     end
183     exportgraphics(f, fullfile(figDir, 'communication_robustness.png'), 'Resolution', 180);
184     closeIfHidden(f, showVisualization);
185 end
186
187 function plotSensitivity(figDir, x, coverage, coverageCI, completion, policies, ...
188     xLabel, fileName, showVisualization)
189 f = figure('Visible', visibilityValue(showVisualization), 'Color', 'w', ...
190     'Position', [100 100 920 400], 'Name', xLabel, 'NumberTitle', 'off');
191 colors = lines(numel(policies));
192 subplot(1, 2, 1); hold on;
193 for pi = 1:numel(policies)
194     errorbar(x, coverage(:, pi), coverageCI(:, pi), '-o', ...
195         'LineWidth', 1.5, 'Color', colors(pi, :));
196 end
197 grid on; xlabel(xLabel); ylabel('覆盖率/%'); ylim([0 100]); styleAxes(gca);
198 subplot(1, 2, 2); hold on;
199 for pi = 1:numel(policies)
200     plot(x, completion(:, pi), '-o', 'LineWidth', 1.5, 'Color', colors(pi, :));
201 end
202 grid on; xlabel(xLabel); ylabel('任务完成率/%'); ylim([0 100]); styleAxes(gca);
203 lgd = legend({'随机', '全局贪心', '分区贪心', 'Q-learning'}, 'Location', 'best');
204 set(lgd, 'Color', 'w', 'TextColor', 'k', 'EdgeColor', [0.25 0.25 0.25]);
205 exportgraphics(f, fullfile(figDir, fileName), 'Resolution', 180);
206 closeIfHidden(f, showVisualization);
207 end
208
209 function value = visibilityValue(showVisualization)
210 if showVisualization
211     value = 'on';
212 else
213     value = 'off';
214 end
215 end
216
217 function closeIfHidden(f, showVisualization)
218 if ~showVisualization
219     close(f);
220 end
221 end
222
223 function styleAxes(ax)
224 set(ax, 'Color', 'w', 'XColor', 'k', 'YColor', 'k', 'GridColor', [0.75 0.75 0.75]);
225 set(ax.Title, 'Color', 'k');
226 set(ax.XLabel, 'Color', 'k');
227 set(ax.YLabel, 'Color', 'k');

```

```

228 end
229
230 function writeMetricHeader(fid, prefix)
231 fprintf(fid, [prefix, ',final_coverage,final_coverage_std,final_coverage_ci95,' ...
232     'completion_rate,completion_rate_ci95,finish_steps,path_length,repeat_rate,' ...
233     'min_agent_distance,conflict_attempts,safety_intervention_rate,reachable_fraction\n']);
234 end
235
236 function writeMetricValues(fid, r)
237 fprintf(fid, '%.6f,%.6f,%.6f,%.6f,%.6f,%.2f,%.2f,%.6f,%.3f,%.2f,%.6f,%.6f\n', ...
238     r.finalCoverage, r.finalCoverageStd, r.finalCoverageCI95, ...
239     r.completionRate, r.completionRateCI95, r.finishSteps, r.pathLength, ...
240     r.repeatRate, r.minDistance, r.collisions, r.safetyRate, r.reachableFraction);
241 end

```

Listing 20 四组扩展实验 runExperiments.m

```

1 function writeOutputs(rootDir, results, policies, seedResults, trainSeeds)
2 %WRITEOUTPUTS Export baseline and training-seed metrics as CSV files.
3 writeMetrics(rootDir, results, policies);
4 writeTrainingSeedMetrics(rootDir, seedResults, trainSeeds);
5 end
6
7 function writeMetricHeader(fid, prefix)
8 fprintf(fid, [prefix, ',final_coverage,final_coverage_std,final_coverage_ci95,' ...
9     'completion_rate,completion_rate_ci95,finish_steps,path_length,repeat_rate,' ...
10     'min_agent_distance,conflict_attempts,safety_intervention_rate,reachable_fraction\n']);
11 end
12
13 function writeMetricValues(fid, r)
14 fprintf(fid, '%.6f,%.6f,%.6f,%.6f,%.6f,%.2f,%.2f,%.6f,%.3f,%.2f,%.6f,%.6f\n', ...
15     r.finalCoverage, r.finalCoverageStd, r.finalCoverageCI95, ...
16     r.completionRate, r.completionRateCI95, r.finishSteps, r.pathLength, ...
17     r.repeatRate, r.minDistance, r.collisions, r.safetyRate, r.reachableFraction);
18 end
19
20 function writeTrainingSeedMetrics(rootDir, seedResults, trainSeeds)
21 fid = fopen(fullfile(rootDir, 'training_seed_metrics.csv'), 'w');
22 writeMetricHeader(fid, 'training_seed');
23 for si = 1:numel(trainSeeds)
24     fprintf(fid, '%d,', trainSeeds(si));
25     writeMetricValues(fid, seedResults{si});
26 end
27 fclose(fid);
28 end
29
30 function writeMetrics(rootDir, results, policies)
31 outFile = fullfile(rootDir, 'simulation_metrics.csv');
32 fid = fopen(outFile, 'w');
33 fprintf(fid, ['policy,final_coverage,final_coverage_std,finish_steps,finish_steps_std,' ...
34     'completion_rate,path_length,path_length_std,conflict_attempts,conflict_attempts_std,' ...
35     'repeat_rate,repeat_rate_std,min_agent_distance,min_agent_distance_std,' ...
36     'safety_intervention_rate,safety_intervention_rate_std,' ...
37     'reachable_fraction,reachable_fraction_std\n']);

```

```

38 for i = 1:numel(policies)
39     r = results.(policies{i});
40     fprintf(fid, ['%.6f,%.6f,%.6f,%.2f,%.2f,%.6f,%.2f,%.2f,%.2f,%.2f,' ...
41         '%.6f,%.6f,%.3f,%.3f,%.6f,%.6f,%.6f,%.6f\n'], ...
42         policies{i}, r.finalCoverage, r.finalCoverageStd, r.finishSteps, r.finishStepsStd, ...
43         r.completionRate, r.pathLength, r.pathLengthStd, r.collisions, r.collisionsStd, ...
44         r.repeatRate, r.repeatRateStd, r.minDistance, r.minDistanceStd, ...
45         r.safetyRate, r.safetyRateStd, r.reachableFraction, r.reachableFractionStd);
46 end
47 fclose(fid);
48 end

```

Listing 21 CSV 指标输出 writeOutputs.m

```

1 function plotBaseline(figDir, trainReward, trainCoverage, results, policies, cfg,
    showVisualization)
2 %PLOTBASELINE Save training, coverage, metric and trajectory figures.
3 plotTraining(figDir, trainReward, trainCoverage, showVisualization);
4 plotCoverage(figDir, results, policies, showVisualization);
5 plotMetrics(figDir, results, policies, showVisualization);
6 plotTrajectories(figDir, cfg, results, showVisualization);
7 end
8
9 function plotTraining(figDir, trainReward, trainCoverage, showVisualization)
10 f = figure('Visible', visibilityValue(showVisualization), 'Color', 'w', 'Position', [80 520 900
    360], ...
11     'Name', '训练曲线', 'NumberTitle', 'off');
12 subplot(1, 2, 1);
13 plot(movmean(trainReward, 20), 'LineWidth', 1.5);
14 grid on; xlabel('训练回合'); ylabel('滑动平均奖励');
15 title('训练奖励');
16 styleAxes(gca);
17 subplot(1, 2, 2);
18 plot(movmean(trainCoverage, 20), 'LineWidth', 1.5);
19 grid on; xlabel('训练回合'); ylabel('覆盖率');
20 title('训练覆盖率');
21 styleAxes(gca);
22 exportgraphics(f, fullfile(figDir, 'training_curve.png'), 'Resolution', 180);
23 closeIfHidden(f, showVisualization);
24 end
25
26 function plotCoverage(figDir, results, policies, showVisualization)
27 f = figure('Visible', visibilityValue(showVisualization), 'Color', 'w', 'Position', [1000 500 780
    430], ...
28     'Name', '覆盖率对比', 'NumberTitle', 'off');
29 hold on;
30 colors = lines(numel(policies));
31 for i = 1:numel(policies)
32     plot(results.(policies{i}).coverageCurve, 'LineWidth', 1.8, 'Color', colors(i, :));
33 end
34 grid on; xlabel('决策步'); ylabel('覆盖率');
35 lgd = legend({'随机', '全局贪心', '分区贪心', 'Q-learning'}, 'Location', 'southeast');
36 set(lgd, 'Color', 'w', 'TextColor', 'k', 'EdgeColor', [0.25 0.25 0.25]);
37 title('覆盖率对比');

```

```

38 ylim([0 1]);
39 styleAxes(gca);
40 exportgraphics(f, fullfile(figDir, 'coverage_comparison.png'), 'Resolution', 180);
41 closeIfHidden(f, showVisualization);
42 end
43
44 function plotMetrics(figDir, results, policies, showVisualization)
45 vals = zeros(numel(policies), 4);
46 for i = 1:numel(policies)
47     r = results.(policies{i});
48     vals(i, :) = [r.finishSteps, r.pathLength, r.repeatRate * 100, r.safetyRate * 100];
49 end
50 f = figure('Visible', visibilityValue(showVisualization), 'Color', 'w', 'Position', [80 20 940
    500], ...
51     'Name', '性能指标', 'NumberTitle', 'off');
52 names = {'完成步数', '总路径长度/栅格', '重复访问率/%', '安全干预率/%'};
53 for k = 1:4
54     subplot(2, 2, k);
55     bar(vals(:, k));
56     grid on; set(gca, 'XTickLabel', {'随机', '全局贪心', '分区贪心', 'Q-learning'});
57     title(names{k});
58     styleAxes(gca);
59 end
60 exportgraphics(f, fullfile(figDir, 'metrics_comparison.png'), 'Resolution', 180);
61 closeIfHidden(f, showVisualization);
62 end
63
64 function plotTrajectories(figDir, cfg, results, showVisualization)
65 obstacles = results.qlearn.sampleObstacles;
66 f = figure('Visible', visibilityValue(showVisualization), 'Color', 'w', 'Position', [1040 20 760
    680], ...
67     'Name', 'Q-learning轨迹', 'NumberTitle', 'off');
68 imagesc(obstacles); axis equal tight; set(gca, 'YDir', 'normal');
69 colormap([1 1 1; 0.15 0.15 0.15]);
70 hold on; grid on;
71 styleAxes(gca);
72 traj = results.qlearn.sampleTraj;
73 colors = lines(cfg.numAgents);
74 for a = 1:cfg.numAgents
75     xs = squeeze(traj(:, a, 1));
76     ys = squeeze(traj(:, a, 2));
77     plot(xs, ys, '-', 'LineWidth', 1.8, 'Color', colors(a, :));
78     plot(xs(1), ys(1), 'o', 'MarkerFaceColor', colors(a, :), 'MarkerEdgeColor', 'k', 'MarkerSize',
    7);
79     plot(xs(end), ys(end), 's', 'MarkerFaceColor', colors(a, :), 'MarkerEdgeColor', 'k', '
    MarkerSize', 7);
80 end
81 xlabel('X/栅格'); ylabel('Y/栅格');
82 title('Q-learning策略样例轨迹');
83 lgd = legend({'无人机1', '起点1', '终点1', '无人机2', '起点2', '终点2', '无人机3', '起点3', '终点3'},
    ...
84     'Location', 'eastoutside');
85 set(lgd, 'Color', 'w', 'TextColor', 'k', 'EdgeColor', [0.25 0.25 0.25]);
86 exportgraphics(f, fullfile(figDir, 'qlearn_trajectories.png'), 'Resolution', 180);
87 closeIfHidden(f, showVisualization);

```

```

88 end
89
90 function value = visibilityValue(showVisualization)
91 if showVisualization
92     value = 'on';
93 else
94     value = 'off';
95 end
96 end
97
98 function closeIfHidden(f, showVisualization)
99 if ~showVisualization
100     close(f);
101 end
102 end
103
104 function styleAxes(ax)
105 set(ax, 'Color', 'w', 'XColor', 'k', 'YColor', 'k', 'GridColor', [0.75 0.75 0.75]);
106 set(ax.Title, 'Color', 'k');
107 set(ax.XLabel, 'Color', 'k');
108 set(ax.YLabel, 'Color', 'k');
109 end

```

Listing 22 基准结果绘图 plotBaseline.m

```

1 function animateSearch(figDir, cfg, result, saveAnimation)
2 % Replay one evaluation episode and optionally save it as an animated GIF.
3 obstacles = result.sampleObstacles;
4 traj = result.sampleTraj;
5 numFrames = size(traj, 1);
6 visited = false(cfg.N, cfg.N);
7 colors = lines(cfg.numAgents);
8 gifFile = fullfile(figDir, 'qlearn_search_process.gif');
9 f = figure('Color', 'w', 'Position', [420 120 820 720], ...
10     'Name', 'Q-learning cooperative search process', 'NumberTitle', 'off');
11
12 for t = 1:numFrames
13     pos = squeeze(traj(t, :, :));
14     for a = 1:cfg.numAgents
15         visited(pos(a, 1), pos(a, 2)) = true;
16     end
17
18     clf(f);
19     ax = axes(f);
20     stateMap = zeros(cfg.N, cfg.N);
21     stateMap(visited) = 1;
22     stateMap(obstacles) = 2;
23     imagesc(ax, stateMap');
24     axis(ax, 'equal', 'tight');
25     set(ax, 'YDir', 'normal', 'XTick', 1:cfg.N, 'YTick', 1:cfg.N);
26     colormap(ax, [1 1 1; 0.78 0.90 0.82; 0.15 0.15 0.15]);
27     clim(ax, [0 2]);
28     hold(ax, 'on'); grid(ax, 'on');
29

```

```

30     for a = 1:cfg.numAgents
31         history = reshape(traj(1:t, a, :), t, 2);
32         plot(ax, history(:, 1), history(:, 2), '-', 'Color', colors(a, :), 'LineWidth', 1.4);
33         plot(ax, pos(a, 1), pos(a, 2), 'o', 'MarkerSize', 11, ...
34             'MarkerFaceColor', colors(a, :), 'MarkerEdgeColor', 'k', 'LineWidth', 1.1);
35         text(ax, pos(a, 1), pos(a, 2), sprintf('%d', a), 'Color', 'w', ...
36             'FontWeight', 'bold', 'HorizontalAlignment', 'center', 'VerticalAlignment', 'middle')
37     ;
38     end
39
40     coverage = nnz(visited & ~obstacles) / nnz(~obstacles);
41     title(ax, sprintf('Q-learning search replay | step %d/%d | coverage %.1f%%', ...
42         t - 1, numFrames - 1, 100 * coverage));
43     xlabel(ax, 'X'); ylabel(ax, 'Y');
44     styleAxes(ax);
45     drawnow;
46
47     % GIF export is optional because frame encoding is slower than live display.
48     if saveAnimation && (t == 1 || t == numFrames || mod(t, 2) == 0)
49         frame = getframe(f);
50         [rgb, ~] = frame2im(frame);
51         [indexed, map] = rgb2ind(rgb, 256);
52         if t == 1
53             imwrite(indexed, map, gifFile, 'gif', 'LoopCount', inf, 'DelayTime', 0.08);
54         else
55             imwrite(indexed, map, gifFile, 'gif', 'WriteMode', 'append', 'DelayTime', 0.08);
56         end
57     end
58     pause(0.04);
59 end
60 if saveAnimation
61     fprintf('Animated replay written to: %s\n', gifFile);
62 end
63
64 function styleAxes(ax)
65     set(ax, 'Color', 'w', 'XColor', 'k', 'YColor', 'k', 'GridColor', [0.75 0.75 0.75]);
66     set(ax.Title, 'Color', 'k');
67     set(ax.XLabel, 'Color', 'k');
68     set(ax.YLabel, 'Color', 'k');
69 end

```

Listing 23 动态搜索过程 animateSearch.m

附录 2：PaperPass 查重报告
